

C++ 메모리·객체 수명·설계

가상 메모리와 RAII에서 객체지향 설계·테스트 가능한 시스템까지

01 메모리·객체 수명·RAII: C++와 운영체제를 함께 보는 법

02 C++ 메모리와 객체 수명 보완

03 C++ 객체지향 설계와 테스트 가능한 시스템

원본 Markdown 9개의 내용·코드·표·수식을 보존하여 연관 주제별로 편집

title: 메모리·객체 수명·RAII aliases:

tags:

created: 2026-08-01 updated: 2026-08-01

- C++ 메모리 관리
- Object Lifetime and RAII
- C++ Ownership
- CS
- OS
- Cpp
- Memory
- RAII
- Ownership

메모리·객체 수명·RAII: C++와 운영체제를 함께 보는 법

[!summary] 한 문장 정리

운영체제는 프로세스에 가상 주소 공간과 페이지를 제공하고, C++는 그 저장 공간 위에서 객체의 수명과 자원 소유권을 관리한다. 안전한 C++의 핵심은 주소를 직접 관리하는 것이 아니라 소유권을 타입으로 표현하고 RAII 객체의 수명에 자원의 수명을 결합하는 것이다.

관련 노트: 01.프로세스와 스레드 · 02.동시성과 병렬성 · OS 공부

학습 목표

- 가상 주소 공간, 스택, 힙, 저장 기간, 객체 수명, 소유권을 서로 구분한다.
- `new` 가 곧바로 운영체제의 페이지 할당을 뜻하지 않는 이유를 설명한다.
- 포인터와 참조가 객체를 소유하는지 단지 관찰하는지를 API에 드러낸다.
- 생성자·소멸자와 예외 처리를 RAII 관점에서 설명한다.
- 복사·이동·얕은 복사·깊은 복사를 자원 소유권과 연결한다.
- 누수, dangling pointer, double free, buffer overflow, use-after-free를 원인부터 예방까지 설명한다.
- 정렬과 객체 모델을 이해하고 undefined behavior를 피한다.
- `unique_ptr` , `shared_ptr` , `weak_ptr` 를 각각 필요한 경우에만 사용한다.

참고한 PDF와 범위

- C:\Users\user\Desktop\OS\cpp.pdf
- PDF 35~50쪽: 참조자와 포인터
- PDF 51~61쪽: `new` 와 `delete`
- PDF 74~112쪽: 생성자, 복사 생성자, 소멸자, 깊은 복사
- PDF 143~180쪽: 직접 만든 문자열 클래스를 통한 동적 메모리 관리

- PDF 635~710쪽: 우측값, 이동, 스마트 포인터, 순환 참조
- [C:\Users\user\Desktop\OS\ostep.pdf](#)
- PDF 134~142쪽: 메모리 가상화와 주소 공간
- PDF 145~152쪽: 스택/힙, 메모리 관리 API와 흔한 오류
- PDF 248~253쪽: present bit, page fault, page replacement
- [C:\Users\user\Desktop\OS\ocs.pdf](#)
- PDF 325쪽 이후: 메인 메모리, 주소 변환, 페이지 테이블
- PDF 367쪽 이후: 가상 메모리, demand paging, copy-on-write, thrashing

[!note] 자료를 읽을 때의 관점

PDF의 C/C++ 예제는 원리를 보여 주는 자료다. 현대 C++ 코드에서는 직접 `new` / `delete` 를 짚기보다 표준 컨테이너와 스마트 포인터를 우선한다. 또한 책의 전형적인 “스택은 아래로, 힙은 위로 자란다”는 그림은 설명용 관례이지 C++ 표준이나 모든 운영체제가 보장하는 배치는 아니다.

1. 먼저 분리해야 할 세 개념

메모리 버그의 상당수는 다음 세 층을 하나로 생각해서 생긴다.

층	핵심 질문	관리 주체	예
저장 공간(storage)	바이트를 놓을 장소가 있는가?	OS, 런타임, 할당자	가상 페이지, 스택 프레임, 힙 블록
객체 수명(lifetime)	그 바이트에 지금 <code>T</code> 객체가 살아 있는가?	C++ 객체 모델	생성 완료부터 소멸 또는 저장 공간 재사용까지
소유권(ownership)	누가 자원을 최종 정리할 책임이 있는가?	프로그램 설계와 타입	지역 객체, <code>unique_ptr</code> , <code>shared_ptr</code>

저장 공간이 있다고 객체가 살아 있는 것은 아니다. 반대로 객체의 수명이 끝난 직후에도 그 주소의 바이트는 할당자나 스택에 남아 있을 수 있다. 그 값을 읽을 수 있다는 뜻은 아니다.

C++

```
alignas(int) std::byte storage[sizeof(int)]; // int를 둘 수 있는 저장 공간만 존재
int* p = std::construct_at(reinterpret_cast<int*>(storage), 42);
// 이 시점부터 int 객체의 수명이 시작됨
std::destroy_at(p);
// 저장 공간은 남아 있지만 int 객체의 수명은 끝남
```

소유권도 주소 자체에 들어 있지 않다. `T*` 라는 비트열만 보고 그 포인터를 `delete` 해야 하는지, 빌린 것인지, 배열 첫 원소인지 알 수 없다. 그래서 현대 C++에서는 타입과 API로 소유권을 표현한다.

2. 프로세스 주소 공간과 가상 메모리

2.1 가상 주소 공간

사용자 프로그램이 다루는 주소는 일반적으로 **가상 주소**다. 운영체제와 MMU는 각 프로세스가 크고 연속적이며 사적인 주소 공간을 가진 것처럼 보이게 한다. 실제 물리 메모리의 프레임은 흩어져 있어도 된다.

TEXT

높은 가상 주소

스레드별 스택 / guard page	호출 프레임, 자동 변수
mmap 영역, 공유 라이브러리	파일 매핑, 큰 할당, shared library
동적 할당 영역(관념적 heap)	allocator가 관리하는 블록
.bss	0으로 초기화되는 전역/정적 데이터
.data	초기값이 있는 전역/정적 데이터
.rodata	읽기 전용 상수
.text	실행 코드

낮은 가상 주소

이 그림은 대표적 배치다. 실제 순서, 성장 방향, 구간 수, 주소는 ABI와 운영체제에 따라 달라지고 ASLR 때문에 실행할 때 마다 바뀔 수 있다. 큰 동적 할당이 전통적 heap 구간이 아니라 별도 메모리 매핑으로 만들어질 수도 있다.

2.2 가상 메모리가 제공하는 것

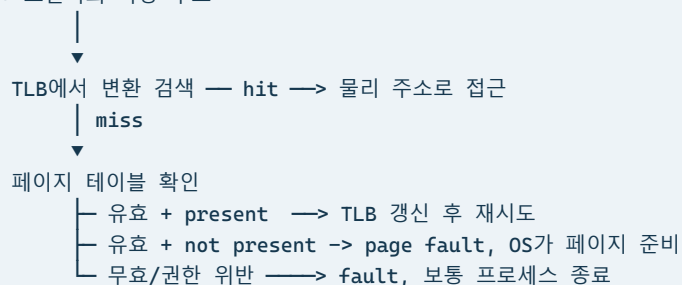
- **격리**: 한 프로세스가 다른 프로세스의 주소 공간을 임의로 읽고 쓰지 못하게 한다.
- **보호**: 페이지마다 읽기, 쓰기, 실행 권한을 둔다.
- **재배치의 투명성**: 프로그램의 가상 주소와 물리 주소를 분리한다.
- **희소한 주소 공간**: 사용하지 않는 구간까지 물리 RAM을 미리 제공할 필요가 없다.
- **공유**: 같은 실행 파일 코드나 공유 라이브러리 페이지를 여러 프로세스가 공유할 수 있다.
- **요구 페이징**: 실제 접근할 때 페이지를 준비할 수 있다.
- **copy-on-write(COW)**: 처음에는 같은 물리 페이지를 읽기 전용으로 공유하고, 쓰기가 생길 때 복사한다.

2.3 주소 변환과 페이지 폴트

가상 주소는 보통 **가상 페이지 번호(VPN) + 페이지 내부 오프셋** 으로 나뉜다. 페이지 테이블은 VPN을 물리 프레임 번호(PFN)와 권한 정보에 대응시킨다. TLB는 최근 주소 변환을 캐시한다.

TEXT

C++ 포인터의 가상 주소



페이지 폴트가 언제나 버그인 것은 아니다. 요구 페이징이나 COW 때문에 발생한 합법적인 폴트는 OS가 처리하고 명령을 재실행한다. 반면 매핑되지 않은 주소나 쓰기 금지 페이지를 접근하면 보통 접근 위반 또는 segmentation fault로 이어진다.

2.4 예약, 커밋, 실제 상주 메모리

“1 GiB를 할당했다”는 말은 다음을 구분해야 한다.

- 가상 주소 범위를 예약했는가?
- OS가 그 범위에 backing store를 약속했는가?
- 실제 페이지를 처음 접근했는가?
- 현재 물리 RAM에 상주하는가?

운영체제 정책에 따라 큰 가상 영역을 얻어도 실제 물리 페이지는 첫 접근 때 준비될 수 있다. 따라서 주소 공간 크기, 커밋 크기, RSS(상주 집합 크기)는 같은 값이 아니다.

2.5 C++ 할당에서 물리 메모리까지

TEXT

new T

- T를 위한 `operator new` 호출
- C++ 런타임/힙 할당자가 `free list` 또는 `arena`에서 블록 검색
- 부족하면 OS에 더 큰 가상 메모리 영역 요청
- T의 생성자 실행
- 최초 페이지 접근 시 필요하면 `page fault`
- OS가 물리 프레임 연결 및 페이지 테이블 갱신

역방향도 단순하지 않다.

TEXT

delete p

- T의 소멸자 실행
- `operator delete`가 블록을 할당자에 반환
- 할당자는 재사용을 위해 보관할 수 있음
- OS에 즉시 페이지를 돌려준다는 보장은 없음

따라서 `delete` 뒤 작업 관리자에서 RSS가 바로 감소하지 않아도 누수라고 단정할 수 없다. 반대로 RSS가 안정적이어도 객체나 자원의 논리적 누수가 없다고 단정할 수 없다.

2.6 OS의 copy-on-write와 C++ 복사는 다른 개념

운영체제의 COW는 가상 메모리 최적화다. 예를 들어 `fork` 직후 부모와 자식의 가상 페이지를 같은 물리 프레임에 읽기 전용으로 연결하고, 어느 한쪽이 쓸 때 `page fault`를 발생시켜 그 페이지를 복사할 수 있다.

이때 C++ 객체별 복사 생성자는 호출되지 않는다. 반대로 `Widget b = a;` 가 실행되어 C++ 복사 생성자가 호출돼도 OS가 반드시 COW `page fault`를 일으키는 것은 아니다.

용어	계층	단위	관찰되는 시점
C++ copy constructor	언어/객체	객체와 그 의미	객체 초기화 표현식
C++ move constructor	언어/객체	소유 상태 이전	우측값으로 새 객체 초기화
OS copy-on-write	가상 메모리	보통 페이지	공유된 페이지에 처음 쓰기

세 기법 모두 “복사 비용을 다룬다”는 공통점만 있을 뿐 계약과 동작 단위가 다르다.

3. Stack과 Heap

3.1 스택

호출 스택은 함수 호출의 실행 문맥을 저장한다. ABI에 따라 반환 주소, 보존 레지스터, 인자, 지역 변수, 정렬용 패딩 등이 스택 프레임에 놓일 수 있다. 최적화된 변수는 레지스터에만 있을 수도 있으므로 “지역 변수는 반드시 물리적인 스택에 있다”는 표현은 지나치게 단순하다.

CPP

```
int square(int x) {
    int result = x * x; // 자동 저장 기간. 실제 배치는 최적화와 ABI에 좌우됨
    return result;
} // result의 수명 종료
```

특징:

- 각 스레드는 독립적인 호출 스택을 가진다.
- 함수 진입과 복귀에 맞춰 저장 공간을 효율적으로 재사용한다.
- 자동 저장 기간 객체는 블록을 벗어날 때 역순으로 파괴된다.
- 무한 재귀나 지나치게 큰 지역 배열은 stack overflow를 일으킬 수 있다.
- 운영체제는 스택 끝에 guard page를 두어 초과 접근을 감지할 수 있다.

3.2 힙과 자유 저장소

일상적으로 C++의 동적 할당 영역을 힙이라고 부른다. 엄밀하게 C++ 표준의 **free store**와 운영체제/할당자 구현의 **heap**은 완전히 같은 표준 용어가 아니다. 실무에서는 **new**가 사용하는 동적 저장 공간을 힙이라고 설명해도 되지만, 구현 세부와 언어 규칙을 구분해야 한다.

CPP

```
auto p = std::make_unique<int>(42); // 동적 저장 기간 객체
// p는 자동 저장 기간을 갖는 소유 핸들
// int 객체는 p가 소유하는 동적 객체
```

힙 할당자는 요청마다 시스템 콜을 하지 않는다. 큰 영역을 OS에서 얻어 작은 블록으로 나누고, 반환된 블록을 free list나 size class에 보관해 재사용한다. 이 과정에서 내부/외부 단편화와 메타데이터 비용이 발생할 수 있다.

3.3 비교

항목	스택/자동 객체	힙/동적 객체
수명 결정	블록과 객체 규칙에 의해 결정적	소유 객체가 해제할 때까지
관리	컴파일러가 생성한 코드와 ABI	할당자 + 프로그램의 소유권 설계
크기	스레드 스택 제한의 영향	주소 공간과 시스템 메모리의 영향
할당 비용	대개 스택 포인터 조정	크기 분류, free list, 동기화 등이 필요할 수 있음
주소 안정성	블록 종료 시 즉시 무효	해제 전까지 존재하지만 컨테이너 이동 등은 별개
대표 위험	반환된 지역 주소, stack overflow	누수, UAF, double free, 단편화

[!important] 흔한 오해

스택 객체가 무조건 빠르고 힙 객체가 무조건 느린 것은 아니다. 성능은 할당 횟수, 캐시 지역성, 객체 크기, 할당자, 스레드 경쟁, 페이지 폴트에 좌우된다. 우선 수명과 소유권에 맞는 설계를 하고 측정으로 최적화한다.

3.4 스택에 들지 힙에 들지

- 작고 수명이 현재 범위에 한정되면 값 또는 지역 객체가 자연스럽다.
- 선택적 값은 먼저 `std::optional<T>` 를 고려한다.
- 가변 길이 연속 데이터는 `std::vector<T>` , 문자열은 `std::string` 을 고려한다.
- 동적 다형성과 단독 소유권이 필요하면 `std::unique_ptr<Base>` 가 자연스럽다.
- 객체가 매우 크거나 주소 안정성이 필요하거나 범위를 넘어 소유권을 이전해야 하면 동적 저장이 유용하다.
- 단지 “복사를 피하려고” 모든 객체를 힙에 두지 않는다. 이동과 copy elision이 있다.

3.5 new/delete 와 malloc/free 를 정확히 구분하기

연산	저장 공간	객체 생성/소멸	실패 방식
<code>new T(args...)</code>	<code>operator new</code> 로 확보	<code>T</code> 생성자 실행	기본적으로 <code>std::bad_alloc</code> 또는 생성자 예외
<code>delete p</code>	소멸 후 <code>operator delete</code> 에 반환	<code>T</code> 소멸자 실행	정상적인 소멸자는 예외를 내보내지 않아야 함
<code>malloc(n)</code>	정렬된 바이트 블록 확보	C++ 클래스 생성자 실행 안 함	실패 시 null pointer
<code>free(p)</code>	<code>malloc</code> 계열 블록 반환	C++ 클래스 소멸자 실행 안 함	잘못된 포인터면 UB

`new` 표현식은 **저장 공간 확보와 객체 생성**을 합친다. 저장 공간 확보 후 생성자가 예외를 던지면 대응하는 deallocation function이 호출되어 확보한 공간을 반환하므로, 성공한 포인터를 받기도 전에 그 블록이 새지 않는다.

CPP

```
auto* a = new int;    // 기본 초기화: 값이 초기화되지 않음
auto* b = new int{};  // 값 초기화: 0

delete a;
delete b;
delete nullptr;      // 아무 동작도 하지 않음
```

배열은 `new T[n]` 과 `delete[]` 가 짝이다. 구현은 원소 수나 파괴 정보를 위한 메타데이터를 별도로 둘 수 있으므로 `delete` 와 바꿔 쓸 수 없다. 현대 C++에서는 이 저수준 짝을 직접 쓰기보다 `vector<T>` , `make_unique<T>()` , `make_unique<T[]>(n)` 을 사용한다.

`new (std::nothrow) T` 는 할당 실패 시 null pointer를 반환하도록 요청할 수 있지만, 생성자가 던지는 예외까지 모두 null로 바꾸는 장치는 아니다. 일반적으로 예외 기반 기본 `new` 와 RAII를 일관되게 사용하는 편이 단순하다.

4. 저장 기간과 객체 수명

4.1 네 가지 저장 기간

저장 기간	대표 예	저장 공간이 유지되는 기간
automatic	지역 변수, 함수 인자	블록 또는 호출과 연관
static	전역 변수, 함수 내부 <code>static</code>	프로그램 실행 기간
thread	<code>thread_local</code>	해당 스레드 실행 기간
dynamic	<code>new</code> , 할당자가 제공한 저장 공간	명시적으로 해제될 때까지

저장 기간은 **저장 공간**에 관한 분류다. 객체 수명은 그 저장 공간 안의 특정 타입 객체가 유효한 기간이다.

4.2 수명의 시작과 끝

일반적인 클래스 객체의 수명은 필요한 초기화가 끝나고 생성자가 성공적으로 완료된 뒤 시작한다. 수명은 다음과 같은 때 끝난다.

- 비클래스 객체가 파괴될 때
- 클래스 객체의 소멸자 호출이 시작될 때
- 저장 공간이 해제되거나 다른 객체에 재사용될 때

생성자가 예외를 던지면 그 객체 자체의 소멸자는 호출되지 않는다. 아직 완성된 객체가 아니기 때문이다. 그러나 이미 생성된 기반 클래스와 멤버 객체는 자동으로 파괴된다. 그래서 멤버 자체가 RAII 객체라면 부분 생성 실패도 안전하다.

4.3 생성과 소멸 순서

생성 순서:

1. 가상 기반 클래스
2. 직접 기반 클래스
3. 멤버가 **클래스에 선언된 순서대로**
4. 생성자 본문

소멸은 정확히 역순이다. 생성자 초기화 목록에 적은 순서가 멤버 생성 순서를 바꾸지 않는다.

CPP

```
class Session {
    Logger logger_;    // 먼저 생성
    Connection conn_; // 나중 생성
public:
    Session() : conn_{}, logger_{} {} // 실제 순서는 logger_ → conn_
}; // 소멸 순서는 conn_ → logger_
```

컴파일러 경고를 켜면 초기화 순서 착각을 발견하는 데 도움이 된다.

4.4 임시 객체

임시 객체는 보통 그것을 포함하는 full-expression의 끝에서 파괴된다.

CPP

```
std::string make_name();

const std::string& ok = make_name();
// 이 직접 바인딩에서는 임시 string의 수명이 ok의 수명까지 연장됨

std::string_view bad = make_name();
// string_view는 소유하지 않음. 문장이 끝나면 임시 string이 파괴되어 bad는 dangling
```

참조를 반환한다고 임시나 지역 객체의 수명이 연장되지는 않는다.

CPP

```
const std::string& wrong() {
    std::string local = "dead soon";
    return local; // 반환 즉시 local 파괴: dangling reference
}
```

4.5 정적 객체의 초기화와 종료

서로 다른 번역 단위에 있는 정적 객체의 동적 초기화 순서는 의존하기 어렵다. 이를 static initialization order fiasco라고 부른다. 필요할 때 생성되는 함수 지역 **static** 으로 의존성을 국소화할 수 있다.

CPP

```
Registry& registry() {
    static Registry instance;
    return instance;
}
```

C++11부터 함수 지역 정적 객체의 최초 초기화는 스레드 안전하다. 다만 종료 시 다른 정적 객체를 소멸자에서 참조하는 순서 문제는 여전히 설계해야 한다.

4.6 다형적 기반 클래스의 소멸자

기반 포인터로 파생 객체를 삭제한다면 기반 클래스의 소멸자는 **virtual** 이어야 한다.

CPP

```

struct Base {
    virtual ~Base() = default;
};

struct Derived final : Base {
    std::vector<int> data;
};

std::unique_ptr<Base> p = std::make_unique<Derived>();
// Base의 virtual destructor를 통해 Derived까지 올바르게 파괴

```

4.7 이미 확보한 저장 공간에 객체 만들기

allocator, arena, 컨테이너 구현에서는 저장 공간 확보와 객체 생성을 분리한다. C++20에서는 `std::construct_at` 과 `std::destroy_at` 이 의도를 잘 드러낸다.

CPP

```

void* raw = ::operator new(sizeof(Widget), std::align_val_t{alignof(Widget)});
Widget* widget = nullptr;
static_assert(std::is_nothrow_destructible_v<Widget>);

try {
    widget = std::construct_at(static_cast<Widget*>(raw));
    use(*widget);
    std::destroy_at(widget); // 객체 수명 종료
    widget = nullptr;
    ::operator delete(raw, std::align_val_t{alignof(Widget)}); // 저장 공간 반환
} catch (...) {
    if (widget != nullptr) {
        std::destroy_at(widget);
    }
    ::operator delete(raw, std::align_val_t{alignof(Widget)});
    throw;
}

```

위 코드는 두 계층을 보여 주기 위한 저수준 예시다. 실전에서는 생성 성공 여부까지 소유하는 RAII 래퍼 또는 allocator-aware 컨테이너를 사용한다. 핵심은 다음 두 동작이 별개라는 점이다.

- `destroy_at` : 객체의 수명을 끝내지만 저장 공간은 남긴다.
- `operator delete` : 저장 공간을 반환하지만 그 전에 살아 있는 비자명 객체를 알아서 파괴하지 않는다.

같은 저장 공간을 다른 타입이나 새 객체에 재사용하면 이전 객체를 가리키던 포인터를 무조건 새 객체 접근에 사용할 수 있는 것은 아니다. 투명하게 교체 가능한 조건과 `std::launder` 가 필요한 경우가 있으므로, 이 수준의 코드는 표준 컨테이너-allocator 구현처럼 제한된 경계 안에 둔다.

5. Pointer와 Reference

5.1 핵심 비교

속성	T*	T&
반드시 즉시 대상 필요	아니오	예
빈 상태 표현	<code>nullptr</code>	정상적인 언어 사용에서는 없음
다른 대상을 다시 가리킴	가능	불가능; 대입은 대상 객체에 대입
포인터 산술	가능	불가능
소유권 표현	자체로 표현하지 못함	소유하지 않음
대표 의미	선택적 관찰, 배열 위치, 저수준 핸들	반드시 존재하는 빌린 객체

참조도 대상보다 오래 살아남으면 dangling이 된다. 구현이 참조를 포인터처럼 표현할 수 있지만, 언어 차원에서 참조는 “널이 될 수 있는 포인터”가 아니다. 널 포인터를 역참조해 참조를 만드는 것부터 undefined behavior다.

5.2 const 의 위치

CPP

```
const int* p1;          // 가리키는 int를 p1을 통해 변경하지 않음
int* const p2 = ...;    // p2 자체를 다른 주소로 바꿀 수 없음
const int* const p3 = ...; // 둘 다 const
```

`const T&` 는 읽기 전용 관찰을 표현하며 불필요한 복사를 피한다. 그러나 그 참조를 보관한다면 호출자가 넘긴 객체의 수명이 충분한지 별도 계약이 필요하다.

5.3 포인터는 소유권이 아니다

CPP

```
void draw(const Widget& widget); // 반드시 존재, 빌림
void draw(const Widget* widget); // nullptr 가능, 빌림
void consume(std::unique_ptr<Widget> widget); // 소유권을 넘겨받음
void share(std::shared_ptr<Widget> widget);    // 공동 소유에 참여
```

함수 인자로 받은 raw pointer를 근거 없이 `delete` 하면 안 된다. 외부 API가 명시적으로 소유권 이전을 계약한 경우가 아니라면 raw pointer/reference는 관찰자라고 간주하는 설계가 안전하다.

5.4 배열과 뷰

포인터 하나에는 길이 정보가 없다. 연속 범위를 전달할 때는 `std::span<T>` 가 포인터와 길이를 함께 전달한다.

CPP

```
int sum(std::span<const int> values) {
    return std::accumulate(values.begin(), values.end(), 0);
}
```

`span`, `string_view`, 반복자는 모두 비소유 뷰다. 원본 객체 파괴뿐 아니라 `vector` 재할당, `string` 변경처럼 저장 위치가 바뀌어도 무효화될 수 있다.

6. RAII

RAII(Resource Acquisition Is Initialization)는 **자원을 획득한 성공적인 객체의 수명과 자원의 유효 기간을 결합**하는 C++ 설계 원칙이다.

1. 생성자 또는 팩터리가 자원을 획득한다.
2. 객체가 유효한 동안 불변식을 유지한다.
3. 소멸자가 자원을 정확히 한 번 반환한다.
4. 복사와 이동 정책이 소유권 의미를 따른다.

자원은 메모리만 뜻하지 않는다. 파일, 소켓, 뮤텝스 잠금, 데이터베이스 트랜잭션, GPU 버퍼, 스레드 조인 책임도 포함한다.

6.1 예외와 조기 반환에 강한 이유

CPP

```
void update(Account& account) {
    std::lock_guard lock{account.mutex()};
    // 이 아래에서 return하거나 예외가 발생해도 lock의 소멸자가 unlock
    account.apply_interest();
}
```

명시적 `lock()` / `unlock()` 사이에 여러 반환 경로가 있으면 해제를 빠뜨리기 쉽다. RAII 객체는 스택 되감기(stack unwinding) 과정에서 파괴되므로 각 경로에 정리 코드를 복제하지 않는다.

6.2 파일 RAII

CPP

```
#include <cstdio>
#include <memory>
#include <stdexcept>

struct FileCloser {
    void operator()(std::FILE* file) const noexcept {
        if (file != nullptr) {
            std::fclose(file);
        }
    }
};

using File = std::unique_ptr<std::FILE, FileCloser>;

File open_file(const char* path, const char* mode) {
    File file{std::fopen(path, mode)};
    if (!file) {
        throw std::runtime_error{"failed to open file"};
    }
    return file; // 이동 또는 copy elision
}
```

`File` 은 복사할 수 없고 이동할 수 있다. 반환 경로, 예외, 정상 종료 모두에서 열린 파일은 소유 객체가 파괴될 때 닫힌다.

6.3 소멸자 원칙

- 소멸자는 일반적으로 예외를 밖으로 던지면 안 된다.
- 이미 stack unwinding 중인데 소멸자가 다시 예외를 던지면 `std::terminate` 로 이어질 수 있다.
- 실패 가능한 “마무리/커밋”은 명시적 멤버 함수로 제공하고, 소멸자는 최소한의 무실패 정리만 수행한다.
- 소멸자는 소유한 멤버의 소멸을 중복 구현하지 않는다. RAII 멤버가 알아서 정리한다.

6.4 RAII와 운영체제의 프로세스 정리는 다르다

프로세스가 정상적으로 범위를 빠져나가면 C++ 소멸자가 결정적인 정리를 한다. 그러나 강제 종료, 전원 차단, `std::abort` , 일부 비정상 종료에서는 모든 자동/정적 객체의 소멸자가 실행된다고 기대할 수 없다.

OS는 프로세스 종료 시 주소 공간의 페이지와 커널 핸들을 회수할 수 있지만, 다음과 같은 **프로그램 의미의 정리**를 대신하지 못한다.

- 파일 형식의 footer 기록
- 데이터베이스 commit/rollback 프로토콜
- 다른 프로세스에 보내야 할 종료 메시지
- 공유 메모리 안의 논리 상태 복구
- 원격 서버의 세션 종료

즉 RAII는 정상적인 C++ 제어 흐름에서 강력하지만, 크래시 일관성은 별도의 프로토콜 문제다.

7. 소유권 모델

7.1 네 가지 역할

역할	의미	대표 타입
값 소유	객체 자체를 직접 가짐	<code>T</code> , <code>vector<T></code> , <code>string</code>
단독 소유	정확히 하나의 소유자가 책임	<code>unique_ptr<T></code>
공동 소유	마지막 소유자가 정리	<code>shared_ptr<T></code>
비소유 관찰	수명을 늘리지 않고 잠시 접근	<code>T&</code> , <code>T*</code> , <code>span</code> , <code>string_view</code> , <code>weak_ptr</code>

기본 선택 순서는 값 → `unique_ptr` → 정말 필요한 경우 `shared_ptr` 다. 소유 관계가 불명확하다는 이유로 `shared_ptr` 부터 사용하면 구조가 더 불명확해진다.

7.2 API 서명으로 의도를 표현하기

서명	일반적인 의미
<code>void f(T value)</code>	복사/이동하여 독립 값을 받음
<code>void f(const T& value)</code>	필수 읽기 전용 빌림
<code>void f(T& value)</code>	필수 수정 가능한 빌림
<code>void f(const T* value)</code>	선택적 읽기 전용 빌림
<code>void f(unique_ptr<T> value)</code>	소유권을 소비함
<code>void f(const unique_ptr<T>&)</code>	포인터 객체를 빌림; 대개 <code>T&</code> 가 더 직접적
<code>void f(shared_ptr<T> value)</code>	공동 소유 기간을 늘림
<code>void f(const shared_ptr<T>&)</code>	<code>shared_ptr</code> 자체를 관찰; 공동 소유 의도가 약하면 <code>T&</code> 검토
<code>weak_ptr<T></code> 보관	공동 소유 그래프의 비소유 링크

7.3 소유권 그래프

트리의 부모가 자식을 `unique_ptr` 로 소유하고 자식이 부모를 raw pointer로 관찰하는 구조는 명확하다.

CPP

```
struct Node {
    Node* parent = nullptr; // 비소유
    std::vector<std::unique_ptr<Node>> children; // 단독 소유

    Node& add_child() {
        auto child = std::make_unique<Node>();
        child->parent = this;
        children.push_back(std::move(child));
        return *children.back();
    }
};
```

`parent` 가 안전하려면 부모가 모든 자식보다 오래 산다는 트리 불변식이 필요하다. 비소유 포인터라고 자동으로 안전해지는 것은 아니다.

8. 복사와 이동

8.1 복사

복사는 원본과 목적지가 이후 독립적으로 유효하다는 의미가 보통 기대된다.

- 복사 생성자: 새 객체를 기존 객체로 초기화
- 복사 대입: 이미 살아 있는 객체의 값을 다른 객체 값으로 바꿈

CPP

```
T b = a; // 복사 생성
b = a;   // 복사 대입
```

8.2 이동

이동은 원본의 자원을 새 객체로 이전해 비싼 복사를 피할 기회를 준다.

CPP

```
std::vector<int> source{1, 2, 3};
std::vector<int> target = std::move(source);
```

`std::move` 는 실제 이동 작업을 하지 않는다. 인자를 우측값으로 취급할 수 있게 캐스팅할 뿐이다. 그 뒤 선택된 이동 생성자/대입 연산자가 자원을 이전한다.

이동된 표준 라이브러리 객체는 보통 **유효하지만 값은 미지정(valid but unspecified)** 상태다. 소멸하거나 새 값을 대입할 수 있지만, 비어 있다고 가정하지 않는다. 특정 타입이 더 강한 사후 조건을 문서화하면 그 계약을 따른다.

8.3 이동 연산과 noexcept

`vector` 가 재할당할 때 기존 원소를 새 저장 공간으로 옮겨야 한다. 이동 생성자가 예외를 던질 수 있으면 강한 예외 보장을 지키기 위해 복사를 선택할 수 있다. 자원 핸들의 이동이 실제로 실패하지 않는다면 `noexcept` 로 선언하는 것이 중요하다.

CPP

```
Handle(Handle&&) noexcept = default;
Handle& operator=(Handle&&) noexcept = default;
```

8.4 copy elision과 반환

현대 C++에서는 지역 객체를 값으로 반환하는 것이 자연스럽다.

CPP

```
std::vector<int> make_data() {
    std::vector<int> result(1'000);
    return result; // NRVO 가능. 불필요한 std::move를 쓰지 않음
}
```

`return std::move(result);` 는 오히려 NRVO를 방해할 수 있다. 소유 객체는 값으로 반환하고 컴파일러의 copy elision과 이동을 활용한다.

8.5 Rule of Zero, Three, Five

- **Rule of Zero:** 자원 관리를 RAII 멤버에 맡기고 특수 멤버 함수를 직접 작성하지 않는다. 가장 권장된다.
- **Rule of Three:** 소멸자, 복사 생성자, 복사 대입 중 하나를 직접 정의해야 한다면 나머지도 검토한다.
- **Rule of Five:** C++11 이후에는 이동 생성자와 이동 대입까지 함께 검토한다.

CPP

```
class Buffer {
    std::vector<std::byte> data_;
public:
    explicit Buffer(std::size_t size) : data_(size) {}
};
// vector가 소멸·복사·이동을 모두 올바르게 처리: Rule of Zero
```

9. 얕은 복사와 깊은 복사

9.1 얕은 복사가 위험한 경우

CPP

```
class BadBuffer {
    std::size_t size_;
    int* data_;
public:
    explicit BadBuffer(std::size_t n)
        : size_{n}, data_{new int[n]{} } {}

    ~BadBuffer() { delete[] data_; }
    // 컴파일러 생성 복사는 data_ 주소만 복사한다.
};
```

CPP

```
BadBuffer a{10};
BadBuffer b = a; // a.data_ == b.data_
```

두 객체가 같은 배열을 자신이 소유한다고 믿는다. 하나가 파괴되면 다른 하나는 dangling pointer를 가지며, 나머지가 파괴될 때 double free가 된다.

9.2 깊은 복사 구현

교육 목적으로 `unique_ptr<T[]>` 를 사용해 독립 복사 의미를 구현하면 다음과 같다.

CPP

```

#include <algorithm>
#include <cstddef>
#include <memory>
#include <stdexcept>
#include <utility>

class Buffer {
    std::size_t size_{};
    std::unique_ptr<int[]> data_;

public:
    explicit Buffer(std::size_t size)
        : size_{size}, data_{std::make_unique<int[]>(size)} {}

    Buffer(const Buffer& other) : Buffer(other.size_) {
        std::copy_n(other.data_.get(), size_, data_.get());
    }

    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            Buffer copy{other};
            swap(copy); // copy-and-swap: 성공한 뒤에만 상태 교체
        }
        return *this;
    }

    Buffer(Buffer&&) noexcept = default;
    Buffer& operator=(Buffer&&) noexcept = default;
    ~Buffer() = default;

    void swap(Buffer& other) noexcept {
        std::swap(size_, other.size_);
        data_.swap(other.data_);
    }

    int& at(std::size_t index) {
        if (index >= size_) {
            throw std::out_of_range{"Buffer::at"};
        }
        return data_[index];
    }
};

```

실무에서는 특별한 요구가 없다면 이 구현보다 `std::vector<int>` 를 멤버로 둔 Rule of Zero 버전이 단순하고 안전하다.

9.3 얇은 복사가 항상 잘못된 아니다

공유가 타입의 의도라면 여러 객체가 같은 상태를 참조하는 복사가 맞을 수 있다. 예를 들어 불변 데이터 공유나 copy-on-write 구현이다. 핵심은 “포인터만 복사했는가”가 아니라 다음이다.

- 복사 후 두 객체의 의미가 독립인가, 공유인가?
- 공유라면 수명은 누가 보장하는가?
- 수정은 공유되는가, 복사되는가?
- 마지막 해제 책임을 어떻게 한 번만 실행하는가?

10. 메모리 오류 분류

10.1 Memory Leak

더 이상 필요하지 않은 자원에 도달할 방법이나 정리 경로를 잃은 상태다.

CPP

```
void leak() {
    auto* p = new int[100];
    p = new int[200]; // 첫 블록 주소를 잃음
    delete[] p;      // 두 번째 블록만 해제
}
```

예방:

- `vector`, `string`, `unique_ptr` 같은 RAII 타입을 사용한다.
- 소유 raw pointer를 만들지 않는다.
- 장기 실행 테스트에서 메모리 사용 추세를 관찰한다.
- AddressSanitizer/LeakSanitizer나 플랫폼 진단 도구를 사용한다.

프로세스 종료 시 OS가 주소 공간을 회수해도 실행 중 누수의 위험과 자원 소멸자의 의미가 사라지는 것은 아니다. `shared_ptr` 순환처럼 포인터가 남아 있어 논리적으로 쓸 수 없는 객체도 누수다.

10.2 Dangling Pointer와 Dangling Reference

대상의 수명이 끝났는데 주소나 참조만 남은 상태다.

CPP

```
int* wrong() {
    int local = 42;
    return &local; // 함수 반환 후 local의 수명 종료
}
```

발생 원인:

- 지역 객체 주소 반환
- `delete` / `free` 후 포인터 보관
- 컨테이너 재할당 뒤 원소 포인터·참조·반복자 사용
- 임시 `string` 에서 만든 `string_view`
- 소유자보다 오래 사는 callback이나 비동기 작업의 참조 캡처

`p = nullptr` 는 같은 포인터의 재사용을 막는 데 일부 도움이 되지만 다른 alias까지 null로 만들지 않는다. 핵심 해결책은 수명 구조를 바꾸는 것이다.

10.3 Use After Free

해제된 동적 저장 공간을 다시 읽거나 쓰는 dangling 사용이다.

CPP

```
int* p = new int{7};
delete p;
// std::cout << *p; // UB: use after free
```

할당자는 그 블록을 이미 다른 객체나 내부 메타데이터로 재사용했을 수 있다. 실행이 “잘 되는 것처럼” 보여도 올바르지 않다. UAF는 제어 데이터나 함수 포인터가 덮일 수 있어 심각한 보안 취약점이 된다.

10.4 Double Free

같은 할당을 두 번 반환하는 오류다.

CPP

```
int* p = new int{1};
int* alias = p;
delete p;
// delete alias; // UB: 같은 블록을 두 번 해제
```

할당자의 free list나 메타데이터가 손상될 수 있다. 얇은 복사된 소유 raw pointer와 서로 독립적으로 만든 `shared_ptr` control block이 대표 원인이다.

10.5 Buffer Overflow

버퍼의 유효 범위를 넘어 쓰는 오류다.

CPP

```
std::array<int, 4> values{};
// values[4] = 10; // UB: 유효 인덱스는 0..3
```

스택 버퍼 overflow와 힙 버퍼 overflow는 저장 위치만 다를 뿐 둘 다 범위 밖 쓰기다. 인접 객체, 반환 주소, 할당자 메타데이터를 손상시킬 수 있다.

예방:

- 포인터와 길이를 따로 전달하지 말고 `span`, `array`, `vector`, `string` 을 쓴다.
- 외부 입력 길이와 산술 overflow를 먼저 검증한다.
- 디버그 시 `.at()` 처럼 범위를 검사하는 인터페이스를 활용한다.
- 문자열의 종료 문자와 바이트/원소 단위를 혼동하지 않는다.

10.6 Invalid Free와 할당 방식 불일치

다음은 모두 잘못이다.

CPP

```
int* a = new int[10];
// delete a;      // new[]와 delete 불일치

int* b = new int;
// std::free(b);   // new와 free 불일치

int* c = a + 3;
// delete[] c;     // 할당 시작 주소가 아님
```

짝은 `new/delete` , `new[]/delete[]` , `malloc/free` 다. 가장 좋은 예방은 이 짝 자체를 사용자 코드에서 없애는 것이다.

10.7 Uninitialized Read

기본 초기화된 일부 스칼라나 `malloc` 으로 얻은 바이트에는 불확정 값이 있을 수 있다. 그런 값을 읽는 것은 오류이며 상황에 따라 undefined behavior가 된다.

CPP

```
int x;           // 자동 지역 int, 값 초기화 안 됨
// int y = x + 1; // 잘못된 읽기

auto p = std::make_unique<int[]>(10); // int 배열은 값 초기화되어 0
```

선언과 동시에 의미 있는 값으로 초기화하고, 메모리를 초기화했는지가 아니라 객체의 불변식이 성립하는지를 확인한다.

10.8 크기 계산 overflow와 범위 착각

버퍼를 할당하기 전 `count * sizeof(T)` 가 정수 범위를 넘으면 실제보다 작은 블록을 얻은 뒤 정상 크기라고 믿고 쓸 수 있다.

CPP

```
std::size_t bytes_for(std::size_t count, std::size_t element_size) {
    if (element_size != 0 &&
        count > std::numeric_limits<std::size_t>::max() / element_size) {
        throw std::length_error{"allocation size overflow"};
    }
    return count * element_size;
}
```

직접 바이트 수를 계산할 이유가 없다면 `vector<T>(count)` 처럼 원소 수를 받는 표준 타입에 맡긴다. 그래도 외부 입력으로 들어오는 `count` 의 업무상 최대치는 먼저 제한해야 한다. “주소 공간에 들어갈 수 있다”와 “서비스가 받아들여야 할 합리적인 크기다”는 다른 조건이다.

11. Memory Alignment

11.1 정렬이 필요한 이유

타입은 주소가 특정 배수에 놓이기를 요구할 수 있다. CPU가 정렬된 접근을 더 효율적으로 처리하거나, 어떤 명령은 잘못 정렬된 접근에서 fault를 발생시킬 수 있다. C++ 객체 모델에서도 요구 정렬을 만족하지 않는 저장 공간에 객체를 잘못 놓고 접근하면 undefined behavior다.

```
CPP
#include <cstdint>

struct Example {
    char tag;    // 1 byte
    int value;   // 보통 더 강한 정렬 필요
};

static_assert(alignof(Example) >= alignof(int));
```

컴파일러는 `tag` 와 `value` 사이 및 객체 끝에 padding을 넣을 수 있다. 끝 padding은 `Example[]` 배열의 각 원소가 올바르게 정렬되도록 한다. 그래서 멤버 크기의 합과 `sizeof(Example)` 이 다를 수 있다.

11.2 alignof 와 alignas

```
CPP
struct alignas(64) CacheLineData {
    std::atomic<std::uint64_t> value{0};
};

static_assert(alignof(CacheLineData) >= 64);
```

`alignas` 는 최소 정렬 요구를 지정한다. 캐시 라인 정렬은 false sharing 완화에 사용할 수 있지만, 실제 하드웨어의 간섭 크기는 `std::hardware_destructive_interference_size` 지원 여부와 측정으로 확인해야 한다. 자세한 병렬성 맥락은 02.동시성과 병렬성을 참고한다.

11.3 동적 할당과 과도 정렬

일반 `new` 는 할당 대상 타입에 필요한 정렬을 만족한다. 과도 정렬(over-aligned) 타입은 정렬 인자를 받는 `operator new` 경로가 사용될 수 있다. 직접 바이트 버퍼나 사용자 정의 allocator를 만들 때는 크기뿐 아니라 정렬과 객체 수명 시작 규칙까지 책임져야 한다.

```
CPP
struct alignas(64) Packet { std::byte data[64]; };
auto packet = std::make_unique<Packet>(); // Packet의 정렬 요구를 만족
```

직렬화된 바이트에서 `reinterpret_cast<T*>` 로 바로 읽는 것은 정렬, 객체 수명, aliasing 문제를 동시에 만들 수 있다. 바이트 표현을 `std::memcpy` 로 적절한 `T` 객체에 복사하는 방법을 우선 검토한다.

12. Undefined Behavior

Undefined behavior(UB)는 표준이 그 상황의 동작을 규정하지 않는다는 뜻이다. “예외가 발생한다”나 “항상 크래시한다”는 뜻이 아니다. 컴파일러는 올바른 프로그램에는 UB가 없다고 가정하고 최적화할 수 있으므로, 디버그 빌드에서 우연히 보인 결과를 신뢰할 수 없다.

대표적인 메모리 관련 UB:

- 배열이나 객체 범위를 벗어난 접근
- 수명이 끝난 객체를 통한 접근, UAF
- 같은 할당의 double delete/free
- 널 포인터 역참조
- 잘못 정렬된 포인터를 통한 객체 접근
- 초기화되지 않은 값의 부적절한 읽기
- `new[]/delete` , `malloc/delete` 같은 불일치
- 유효하지 않은 downcast 또는 잘못된 타입으로 객체 접근
- signed integer overflow
- 동기화 없는 data race

12.1 OS fault와 UB는 같은 개념이 아니다

- UB는 C++ 언어 의미의 문제다.
- page fault/access violation/segmentation fault는 하드웨어·OS가 관찰한 사건이다.

UB가 OS fault로 나타날 수 있지만 반드시 그렇지는 않다. 버퍼를 한 칸 넘겨 썼는데 그 주소가 같은 쓰기 가능 페이지 안이라면 OS는 막지 못한다. 언어적으로는 이미 UB다. 반대로 합법적인 요구 페이지징 page fault는 UB가 아니다.

12.2 “메모리에 값이 아직 남아 있다”는 변명이 안 되는 이유

객체 수명이 끝나도 비트가 즉시 지워지지 않을 수 있다. 그러나 C++ 추상 기계에서 그 객체는 더 이상 존재하지 않는다. 할당자는 언제든지 같은 바이트를 다른 용도로 쓸 수 있고 컴파일러도 수명 규칙을 기반으로 최적화한다. 물리적 잔존과 언어적 유효성은 다르다.

13. 스마트 포인터

13.1 `std::unique_ptr` : 기본 선택

단독 소유권을 표현한다. 복사는 금지되고 이동으로 소유권을 이전한다.

CPP

```
auto first = std::make_unique<Widget>();
auto second = std::move(first);
// first는 이제 소유하지 않음. second가 유일한 소유자
```

장점:

- 비용 모델이 raw pointer와 가깝다.
- 소유자가 명확하다.
- 범위를 벗어날 때 정확히 한 번 삭제한다.
- 사용자 정의 deleter로 파일, 소켓 등에도 사용할 수 있다.

배열에는 `unique_ptr<T[]>` 가 있지만 크기·반복·복사 의미가 필요하면 `vector<T>` 가 대개 더 좋다.

13.2 `std::shared_ptr` : 공동 소유가 실제 요구일 때

`shared_ptr` 복사본들은 control block을 공유한다. control block에는 강한 참조 수, 약한 참조 수, deleter 등의 정보가 있다. 마지막 강한 소유자가 사라지면 객체가 파괴된다.

CPP

```
auto a = std::make_shared<Widget>();
auto b = a; // 같은 객체와 control block 공동 소유
```

주의:

- reference count 갱신 비용과 control block 메모리 비용이 있다.
- 참조 수 조작이 스레드 안전하다는 것과 `Widget` 객체의 멤버 접근이 스레드 안전하다는 것은 별개다.
- 수명 종료 시점이 분산되어 결정성이 약해질 수 있다.
- 소유권 문제를 설계하지 않고 `shared_ptr` 로 덮으면 순환 참조와 장기 보유가 생긴다.

13.3 하나의 객체에 control block을 두 개 만들지 않는다

CPP

```
Widget* raw = new Widget;
std::shared_ptr<Widget> a{raw};
// std::shared_ptr<Widget> b{raw}; // 서로 다른 control block: 마지막에 double delete
```

처음부터 `make_shared` 를 사용하고 그 `shared_ptr` 를 복사한다.

CPP

```
auto a = std::make_shared<Widget>();
auto b = a;
```

객체가 자기 자신의 `shared_ptr` 를 얻어야 한다면 `std::enable_shared_from_this<T>` 와 `shared_from_this()` 를 사용한다. `shared_ptr<T>(this)` 를 새로 만들면 안 된다. 또한 해당 객체가 이미 `shared_ptr` 에 의해 관리되고 있다는 전제도 지켜야 한다.

13.4 `make_shared` 의 장점과 트레이드오프

`make_shared<T>` 는 일반적으로 객체와 control block을 한 번의 할당으로 묶어 지역성과 예외 안전성을 개선한다. 다만 약한 참조가 오래 남으면 객체는 이미 파괴되었더라도 결합된 할당 블록 전체의 반환이 늦어질 수 있다. 매우 큰 객체에서 이것이 중요하다면 별도 할당을 검토할 수 있다.

13.5 std::weak_ptr : 순환을 끊는 관찰자

CPP

```
struct Person {
    std::string name;
    std::shared_ptr<Person> partner;
};
```

두 `Person` 이 서로를 `shared_ptr` 로 가리키면 외부 소유자가 사라져도 참조 수가 0이 되지 않는다. 한쪽 또는 양쪽의 비소유 관계를 `weak_ptr` 로 표현한다.

CPP

```
struct Person {
    std::string name;
    std::weak_ptr<Person> partner; // 관찰만 함

    void greet_partner() const {
        if (auto p = partner.lock()) { // 성공 시 작업 중 수명을 유지
            std::cout << p->name;
        }
    }
};
```

`expired()` 확인 후 별도로 사용하면 그 사이 수명이 끝날 수 있다. `lock()` 결과의 `shared_ptr` 가 비어 있는지 확인하는 방식이 안전하다.

13.6 스마트 포인터가 필요 없는 곳

- 지역 값 객체
- `vector` , `string` , `map` 등 컨테이너가 직접 소유하는 원소
- 수명을 늘리지 않는 함수 인자: `T&` , `const T&` , `T*`
- 다른 객체의 멤버로 직접 포함할 수 있는 경우

모든 raw pointer가 나쁜 것은 아니다. **소유 raw pointer**가 특히 위험하다. 비소유 raw pointer도 수명 계약이 분명해야 한다.

14. 예외 안전성과 트랜잭션적 갱신

보장	의미
no-throw guarantee	연산이 예외를 던지지 않음
strong guarantee	실패하면 관찰 가능한 상태가 연산 전과 같음
basic guarantee	실패해도 불변식과 자원 안전은 유지됨
no guarantee	실패 후 상태나 자원 안전을 보장하지 못함

RAII는 자원 누수를 막는 기반이고, strong guarantee는 상태 변경 순서까지 설계해야 한다.

CPP

```
void Config::replace(std::vector<Entry> next) {
    validate(next);      // 실패 가능 작업을 먼저
    entries_.swap(next); // no-throw commit
}
```

앞의 `Buffer` 복사 대입도 임시 복사에 성공한 뒤 `swap` 하는 copy-and-swap으로 strong guarantee를 제공한다.

15. 성능과 메모리 지역성

안전한 소유권은 성능과 대립하지 않는다. 오히려 값과 컨테이너 중심 설계는 연속 배치와 명확한 수명 때문에 최적화하기 쉽다.

15.1 vector 와 포인터 그래프

`vector<T>` 는 원소를 연속 저장해 캐시 지역성과 prefetch에 유리하다. 각 원소를 개별 `new` 로 만들고 포인터로 연결하면 할당 비용, 단편화, cache miss가 늘 수 있다.

15.2 재할당과 무효화

CPP

```
std::vector<int> values;
values.push_back(1);
int* p = &values[0];
values.push_back(2); // capacity가 부족하면 재할당 가능
// p는 재할당이 있었다면 dangling
```

필요 크기를 알면 `reserve` 로 재할당 횟수를 줄일 수 있지만, reserve 뒤에도 그 capacity를 넘으면 무효화된다. 포인터를 장기 보관하지 않는 설계가 우선이다.

15.3 allocator, pool, arena

많은 작은 객체를 일정한 단계에서 한꺼번에 버린다면 pool/arena가 유용할 수 있다. C++의 `std::pmr` 은 메모리 자원 정책을 컨테이너와 분리한다. 하지만 arena가 객체 소멸자의 의미를 없애는 것은 아니다. 비메모리 자원이나 비자명한 소멸이 있다면 올바른 파괴 순서를 보장해야 한다.

15.4 working set과 thrashing

프로그램의 활성 페이지 집합이 물리 메모리에 맞지 않으면 page fault와 교체가 반복되어 성능이 급락할 수 있다. 이를 thrashing이라고 한다. 메모리 사용량만 줄이는 것뿐 아니라 접근 패턴과 지역성 개선이 중요하다.

16. 도구로 검증하기

컴파일 성공과 한 번의 정상 실행은 메모리 안전의 증거가 아니다.

16.1 컴파일러 경고

MSVC 예:

POWERSHELL

```
cl /std:c++20 /EHsc /W4 /permissive- example.cpp
```

GCC/Clang 예:

BASH

```
c++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion example.cpp
```

16.2 Sanitizer

GCC/Clang 계열 예:

BASH

```
c++ -std=c++20 -g -O1 \
    -fsanitize=address,undefined \
    -fno-omit-frame-pointer example.cpp
```

- AddressSanitizer: out-of-bounds, UAF, 일부 double free
- UndefinedBehaviorSanitizer: 여러 UB 범주
- LeakSanitizer: 누수
- MemorySanitizer: 초기화되지 않은 읽기, 지원 환경 제한
- ThreadSanitizer: data race; AddressSanitizer와 보통 별도 실행

MSVC도 지원 버전에서 `/fsanitize=address` 를 사용할 수 있다. 도구마다 탐지 범위와 플랫폼 제한이 있으므로 여러 구성과 테스트를 조합한다.

16.3 테스트해야 할 경계

- 크기 0, 1, 최대 허용 크기
- 복사 후 원본/복사본 독립 수정
- 자기 대입과 자기 이동을 요구하는지
- 예외가 생성 중간에 발생하는 경우
- 컨테이너 재할당 뒤 핸들 무효화
- 소유자 파괴 전후 callback과 비동기 작업
- 순환 소유 그래프
- 여러 스레드에서 같은 객체를 접근하는 경우

17. 실전 설계 예제

17.1 요청 메시지를 안전하게 처리하기

CPP

```
#include <cstddef>
#include <span>
#include <stdexcept>
#include <vector>

class Message {
    std::vector<std::byte> payload_; // 값 소유, Rule of Zero

public:
    explicit Message(std::span<const std::byte> input)
        : payload_{input.begin(), input.end()} {
        if (payload_.size() > 1'048'576) {
            throw std::length_error{"message too large"};
        }
    }

    std::span<const std::byte> payload() const noexcept {
        return payload_;
    }
};

void inspect(const Message& message) {
    for (std::byte b : message.payload()) {
        (void)b;
    }
}
```

이 설계에서:

- 입력 `span` 은 함수 호출 중 빌림이다.
- `Message` 는 필요한 바이트를 `vector` 로 독립 소유한다.
- 길이가 함께 전달되므로 raw pointer만 전달하는 것보다 범위 계약이 분명하다.
- 소멸, 복사, 이동은 `vector` 에 맡겨 Rule of Zero를 지킨다.

17.2 polymorphic task의 소유권 이전

CPP

```
#include <memory>
#include <utility>
#include <vector>

struct Task {
    virtual ~Task() = default;
    virtual void run() = 0;
};

class Scheduler {
    std::vector<std::unique_ptr<Task>> queue_;
public:
    void submit(std::unique_ptr<Task> task) {
        queue_.push_back(std::move(task));
    }
};
```

`submit` 의 서명만 보아도 호출자가 소유권을 넘긴다는 것을 알 수 있다. 공동 소유 요구가 없으므로 `shared_ptr` 를 사용하지 않는다.

17.3 비동기 작업의 수명

CPP

```
class Worker : public std::enable_shared_from_this<Worker> {
public:
    void start(Executor& executor) {
        auto self = shared_from_this();
        executor.post([self] {
            self->do_work();
        });
    }
private:
    void do_work();
};
```

비동기 callback이 실행될 때까지 객체가 반드시 살아 있어야 한다면 작업이 `shared_ptr` 를 캡처해 수명을 연장할 수 있다. 반대로 작업 때문에 객체가 무한히 살아 있으면 안 된다면 `weak_ptr` 를 캡처하고 실행 시 `lock()` 한다. 어느 쪽이 맞는지는 “작업이 객체를 소유하는가?”라는 설계 질문으로 결정한다.

18. 자주 나오는 질문

Q1. 지역 객체는 항상 스택에 있는가?

언어적으로는 automatic storage duration이라고 말하는 것이 정확하다. 최적화에 따라 레지스터에만 있거나 객체 자체가 제거될 수 있다. 관념적 호출 스택과 실제 기계 배치를 구분한다.

Q2. `new` 는 시스템 콜인가?

보통 아니다. `new` 표현식은 저장 공간을 얻는 `operator new` 와 객체 생성으로 구성된다. 런타임 할당자는 이미 확보한 블록을 반환할 수 있고, 부족할 때만 OS API를 통해 더 큰 영역을 요청한다.

Q3. delete 하면 물리 메모리가 바로 줄어드는가?

보장되지 않는다. 블록이 할당자에 반환되어 같은 프로세스의 다음 할당에 재사용될 수 있다. 큰 매핑이나 할당자 정책에 따라 OS로 일부 반환될 수 있다.

Q4. 프로세스가 끝나면 OS가 다 회수하는데 누수를 왜 신경 쓰는가?

장기 실행 프로세스는 종료 전 메모리가 고갈될 수 있다. 또한 소멸자는 메모리 외 자원의 의미 있는 정리를 수행한다. 테스트 가능성, 조기 해제, 라이브러리 재사용, 종료 절차의 정확성도 중요하다.

Q5. shared_ptr 면 dangling pointer가 사라지는가?

아니다. 그 `shared_ptr` 가 공동 소유하는 객체의 수명만 관리한다. 객체 내부의 raw pointer, 뷰, 외부 자원은 여전히 dangling이 될 수 있다. `get()` 으로 얻은 포인터를 소유자보다 오래 보관해도 위험하다.

Q6. nullptr 로 만들면 UAF가 해결되는가?

한 변수의 실수는 줄일 수 있지만 같은 객체를 가리키는 다른 alias는 그대로다. 소유권을 하나로 만들고 비소유 접근 범위를 짧게 하는 것이 해결책이다.

Q7. 참조는 포인터보다 무조건 안전한가?

참조는 필수 대상과 재바인딩 불가라는 의도를 더 잘 표현하지만 수명은 늘리지 않는다. dangling reference도 UB를 만든다.

Q8. OS가 buffer overflow를 막아 주지 않는가?

페이지 권한 경계를 넘으면 막을 수 있지만 같은 유효 페이지 안에서 인접 객체를 덮는 것은 알지 못한다. C++ 범위 규칙, 안전한 타입, sanitizer가 필요하다.

19. 리뷰 체크리스트

소유권

- ☐ 각 자원에 최종 정리 책임자가 정확히 하나 또는 명시된 공동 소유자로 존재한다.
- ☐ owning raw pointer가 없다.
- ☐ `shared_ptr` 사용 이유가 “여러 곳에서 쓴다”가 아니라 실제 공동 수명 요구다.
- ☐ 순환 참조를 `weak_ptr` 또는 비소유 관계로 끊었다.
- ☐ API 서명에서 소비, 공유, 필수 빌림, 선택적 빌림이 드러난다.

객체 수명

- ☐ 반환하는 reference/view가 지역 또는 임시 객체를 가리키지 않는다.
- ☐ callback과 비동기 작업이 캡처한 대상의 수명이 충분하다.
- ☐ `vector` / `string` 재할당 후 포인터, 참조, 반복자를 재사용하지 않는다.
- ☐ 다형적 기반 포인터로 삭제한다면 소멸자가 virtual이다.
- ☐ 생성자 실패 중 이미 획득한 자원은 RAII 멤버가 정리한다.

복사와 이동

- ☐ 복사가 독립 값인지 공유 값인지 의미가 명확하다.
- ☐ 자원 관리 클래스는 Rule of Zero를 우선한다.
- ☐ 이동 후 원본에 문서화되지 않은 상태를 가정하지 않는다.
- ☐ 실패하지 않는 이동 연산은 `noexcept` 인지 검토한다.
- ☐ 반환 지역 객체에 불필요한 `std::move` 를 쓰지 않는다.

경계와 UB

- ☐ 포인터와 길이가 분리되어 불일치할 수 있는 API를 피한다.
- ☐ 크기 계산의 정수 overflow를 확인한다.
- ☐ 초기화되지 않은 값을 읽지 않는다.
- ☐ `new/delete` , `new[]/delete[]` , `malloc/free` 를 섞지 않는다.
- ☐ 정렬과 실제 객체 수명을 만족하지 않는 타입 punning을 하지 않는다.
- ☐ 공유 메모리 접근에 data race가 없다.

검증

- ☐ 높은 경고 수준으로 빌드했다.
- ☐ 경계값과 예외 경로를 테스트했다.
- ☐ 가능한 플랫폼에서 ASan/UBSan/LSan 또는 동등 도구를 실행했다.
- ☐ 장기 실행에서 메모리와 핸들 증가 추세를 측정했다.

20. 최종 압축 정리

1. 포인터 값은 가상 주소이며 소유권 정보가 아니다.
2. 스택/힙은 저장 공간 관점이고 객체 수명은 C++ 언어 규칙이다.
3. 저장 공간이 남아 있어도 객체의 수명이 끝났다면 접근할 수 없다.
4. `new` 는 객체 생성까지, `delete` 는 객체 소멸 후 저장 공간 반환까지 수행한다.
5. 할당자는 OS와 프로그램 사이에서 블록을 캐시하므로 `delete` 와 RSS 감소는 일대일이 아니다.
6. RAII는 자원 수명을 객체 수명에 결합해 정상 반환, 조기 반환, 예외에서 같은 정리 규칙을 적용한다.
7. 값과 표준 컨테이너를 우선하고 동적 다형성에는 `unique_ptr` 를 기본으로 쓴다.
8. `shared_ptr` 는 실제 공동 소유에만 쓰고 순환은 `weak_ptr` 로 끊는다.
9. 복사는 의미를 복제하고 이동은 자원을 이전한다. `std::move` 자체는 이동하지 않는다.
10. 얇은 복사가 단독 소유 raw pointer를 복제하면 double free와 UAF가 된다.
11. buffer overflow와 UAF는 OS가 항상 감지하지 못하는 C++ UB다.
12. 정렬, 범위, 객체 수명, aliasing 규칙은 최적화 빌드에서도 지켜야 하는 언어 계약이다.
13. 컴파일 성공은 충분하지 않다. 경고, 테스트, sanitizer, 장기 관찰로 검증한다.

[!tip] 가장 실용적인 기본 규칙

직접 `new / delete` 를 쓰지 말고 값, 표준 컨테이너, `unique_ptr` , 범위 뷰를 조합하라. raw pointer와 reference는 짧은 비소유 관찰자로 사용하고, 소유권과 수명 계약을 함수 서명에서 읽을 수 있게 만들어라.

title: C++ 메모리와 객체 수명 보완 aliases:

tags:

created: 2026-08-03 updated: 2026-08-03

- 메모리와 객체 수명 보완
- C++ Memory and Object Lifetime Supplement
- CS
- OS
- Cpp
- Memory
- ObjectLifetime
- Ownership

C++ 메모리와 객체 수명 보완

[!summary] 문서의 역할

이 문서는 00.메모리·객체 수명·RAII를 다시 요약하지 않는다. 세 PDF와 00~03 문서를 항목별로 대조한 뒤, **00번에 없거나 여러 절에 흩어져 있던 연결 지식만 보완한다.**

관련 노트: 00.메모리·객체 수명·RAII · 01.프로세스와 스레드 · 02.동시성과 병렬성 · 03.Mutex와 Condition Variable

1. 중복을 제외한 결과

요청된 기본 항목은 이미 00번 문서에 모두 존재한다. 따라서 다음 표의 내용은 이 문서에서 반복하지 않고 해당 절로 연결한다.

공부 항목	기존 문서 위치
Stack과 Heap	00.메모리·객체 수명·RAII#3. Stack과 Heap
가상 메모리와 프로세스 주소 공간	00.메모리·객체 수명·RAII#2. 프로세스 주소 공간과 가상 메모리
Pointer와 Reference	00.메모리·객체 수명·RAII#5. Pointer와 Reference
객체의 생성·소멸 시점	00.메모리·객체 수명·RAII#4. 저장 기간과 객체 수명
RAII	00.메모리·객체 수명·RAII#6. RAII
소유권	00.메모리·객체 수명·RAII#7. 소유권 모델
복사와 이동	00.메모리·객체 수명·RAII#8. 복사와 이동
얕은 복사와 깊은 복사	00.메모리·객체 수명·RAII#9. 얕은 복사와 깊은 복사
Memory Leak, Dangling Pointer, Double Free	00.메모리·객체 수명·RAII#10. 메모리 오류 분류
Buffer Overflow, Use After Free	00.메모리·객체 수명·RAII#10. 메모리 오류 분류
Memory Alignment	00.메모리·객체 수명·RAII#11. Memory Alignment

공부 항목	기존 문서 위치
Undefined Behavior	00.메모리·객체 수명·RAII#12. Undefined Behavior
스마트 포인터 기본	00.메모리·객체 수명·RAII#13. 스마트 포인터

이 문서에서 새로 보완하는 내용

1. C++ 객체에서 힙 할당기와 OS 페이지까지 이어지는 실제 관리 계층
2. 힙 할당기의 빈 공간 분할, 병합, 내부·외부 단편화
3. TLB hit/miss, 페이지 테이블, present bit, page fault의 관계
4. `unique_ptr`, `shared_ptr`, `weak_ptr`, raw pointer 사이의 소유권 전이 규칙
5. API 선언에서 소유·공유·관찰을 구분하는 실전 예제

2. PDF 대조 범위

PDF	확인 범위	이 문서에서 사용하는 근거
<code>cpp.pdf</code>	PDF 676~708쪽	RAII, <code>unique_ptr</code> 의 이동과 <code>get()</code> , <code>shared_ptr</code> control block, <code>make_shared</code> , 순환 참조, <code>weak_ptr::lock()</code>
<code>ostep.pdf</code>	PDF 182~194쪽	사용자 수준 힙 할당기, 외부·내부 단편화, splitting, coalescing, 적합 정책
<code>ostep.pdf</code>	PDF 248~249쪽	TLB hit/miss, PTE, present bit, page fault handler
<code>ocs.pdf</code>	PDF 325~345쪽	논리·물리 주소 변환, TLB가 있는 페이징, TLB 적중률과 접근 비용
<code>ocs.pdf</code>	PDF 367~375쪽	요구 페이징과 유효·무효 비트, page fault 처리

[!important] 근거 사용 원칙

이 문서의 보완 설명은 위 범위에서 확인되는 내용만 사용한다. C++ 언어 규칙의 전체 명세를 대신하지 않으며, PDF가 다루지 않는 스마트 포인터 API를 임의로 확장해 설명하지 않는다.

3. 메모리를 한 층으로 보면 안 되는 이유

PDF들을 함께 보면 C++ 객체 하나의 메모리는 다음 계층을 거친다.

TEXT

C++ 객체와 객체 수명



`new/delete`와 C++ 런타임



사용자 수준 메모리 할당기와 힙 청크



프로세스 가상 주소 공간



TLB와 페이지 테이블을 통한 주소 변환



물리 프레임 또는 보조 저장장치

각 계층의 질문은 서로 다르다.

계층	핵심 질문
C++ 객체	해당 저장 공간에 지금 T 객체가 살아 있는가?
소유권	누가 객체 또는 자원의 정리를 책임지는가?
할당기	요청을 만족할 빈 청크가 있는가? 청크를 분할·병합해야 하는가?
가상 메모리	가상 페이지가 유효하며 현재 물리 메모리에 존재하는가?
하드웨어 변환	TLB에 변환 정보가 있는가? 페이지 테이블을 읽어야 하는가?

이 질문들은 서로 대체되지 않는다.

- 주소 변환이 성공해도 C++ 객체 수명이 끝났다면 그 객체를 사용하면 안 된다.
- 힙 전체의 빈 공간 합계가 충분해도 연속된 빈 청크가 부족하면 할당이 실패할 수 있다.
- 가상 주소가 유효해도 페이지가 현재 물리 메모리에 없다면 page fault가 발생할 수 있다.
- 스마트 포인터가 객체 수명을 관리해도 객체 내부 범위 검사까지 대신하지는 않는다.

4. 힙 할당기의 빈 공간 관리

4.1 malloc 은 시스템 콜 자체가 아니다

OSTEP은 `malloc()` 과 `free()` 를 사용자 수준 라이브러리 함수로 설명한다. 할당 라이브러리는 프로세스 가상 주소 공간의 일부를 관리하고, 더 큰 영역이 필요할 때 운영체제의 지원을 받는다.

C++의 `new` 표현식도 단순히 "OS에서 페이지 하나를 받는다"는 뜻으로 해석하면 안 된다. 00번의 C++ 할당에서 물리 메모리까지와 연결해 다음처럼 기억한다.

FORMULA · CALCULATION

`new` 표현식

- = 저장 공간 획득 시도
- + 그 저장 공간에 객체 생성

`delete` 표현식

- = 객체 파괴
- + 저장 공간 반환

반환된 저장 공간은 먼저 할당기의 빈 공간으로 돌아갈 수 있다. 이 사실과 객체 수명 종료는 별개의 사건이다.

4.2 할당기의 header

`free(void* ptr)` 는 크기를 별도 인자로 받지 않는다. OSTEP의 할당기 설명에서는 할당된 청크 앞의 header에 크기 같은 정보를 저장하여, 반환 시 해당 영역의 크기를 알아내는 방식을 보여 준다.

TEXT

```
[ allocator header ][ caller가 사용하는 payload ]
                ↑
            ptr
```

이 구조는 다음 오류가 왜 할당기 자체까지 손상시킬 수 있는지 이해하는 데 도움이 된다.

- 같은 포인터를 두 번 해제하는 double free
- 할당된 범위 앞이나 뒤를 덮어쓰는 buffer overflow
- 서로 다른 할당·해제 방식을 잘못 짚는 오류

구체적인 오류 분류와 C++ 예방책은 00.메모리·객체 수명·RAII#10. 메모리 오류 분류에서 다룬다.

4.3 Splitting

빈 청크가 요청 크기보다 크다면 할당기는 청크를 둘로 나눌 수 있다.

TEXT

```
할당 전: [          빈 청크 30          ]
요청 12: [ 사용 12 ][          빈 청크 18          ]
```

요청을 만족한 앞부분은 호출자에게 주고, 나머지는 빈 공간 자료 구조에 남긴다. 이것이 splitting이다.

4.4 Coalescing

서로 인접한 빈 청크를 그대로 따로 두면 큰 요청을 만족하지 못할 수 있다.

TEXT

```
병합 전: [ 빈 10 ][ 빈 10 ][ 빈 10 ]
병합 후: [          빈 30          ]
```

전체 빈 공간이 30이어도 10짜리 청크 세 개로만 인식하면 20짜리 연속 요청은 실패한다. 반환된 청크의 이웃이 비어 있을 때 하나로 합치는 것이 coalescing이다.

4.5 내부 단편화와 외부 단편화

구분	PDF에서 확인되는 의미
내부 단편화	할당기가 요청보다 큰 청크를 제공해 할당된 청크 내부 에 사용하지 않는 공간이 생김
외부 단편화	빈 공간 총량은 충분하지만 여러 작은 청크로 흩어져 필요한 연속 청크 가 없음

따라서 다음 두 문장은 동시에 참일 수 있다.

- 프로세스의 힙에는 빈 공간이 남아 있다.
- 현재 크기의 새 할당 요청은 실패한다.

4.6 적합 정책의 비용

OSTEP과 OCS는 대표적인 연속 할당 정책을 비교한다.

정책	선택	확인되는 특성
Best fit	요청을 만족하는 가장 작은 빈 청크	공간 낭비를 줄이려 하지만 전체 검색 비용이 들 수 있음
Worst fit	가장 큰 빈 청크	큰 잔여 청크를 남기려 하지만 검색 비용과 단편화 문제가 있음
First fit	처음 발견한 충분한 청크	전체를 항상 검색하지 않아 빠를 수 있으나 앞부분에 작은 조각이 모일 수 있음
Next fit	마지막 탐색 위치부터 검색	탐색 시작점을 분산시키고 전체 검색을 피하려는 방식

핵심은 “Heap은 큰 연속 배열이고 `delete` 가 빈칸을 하나 만든다” 정도로 끝나지 않는다는 점이다. 할당기는 청크의 크기와 위치를 추적하고, 분할·병합·탐색 정책을 수행한다.

5. 가상 주소 접근의 실제 경로

5.1 TLB hit

CPU가 가상 주소를 만들면 하드웨어는 가상 페이지 번호(VPN)에 해당하는 변환 정보가 TLB에 있는지 먼저 확인한다.

TEXT

```

가상 주소
  ↓ VPN 추출
TLB 조회 -- hit --> 물리 프레임 번호(PFN)
                      ↓
                      실제 메모리 접근
  
```

TLB hit이면 페이지 테이블을 추가로 읽지 않고 물리 주소를 얻을 수 있다. OCS는 이 적중률이 실질 메모리 접근 시간에 직접 영향을 준다는 계산 예를 제시한다.

5.2 TLB miss

TLB에 변환 정보가 없으면 페이지 테이블에서 PTE를 확인해야 한다.

TEXT

```

TLB miss
  ↓
페이지 테이블에서 PTE 조회
  ↓
PTE가 유효하고 페이지가 메모리에 존재
  ↓
PFN을 TLB에 적재
  ↓
명령 재실행
  
```

TLB miss는 곧바로 “잘못된 포인터”를 뜻하지 않는다. 필요한 주소 변환 정보가 TLB에 없어서 페이지 테이블을 추가로 확인해야 한다는 뜻이다.

5.3 Present bit와 page fault

PTE의 present bit는 해당 페이지가 현재 물리 메모리에 존재하는지를 표현한다.

- present bit가 설정되어 있으면 PTE에서 PFN을 얻어 접근을 계속할 수 있다.
- 페이지가 유효하지만 현재 물리 메모리에 없다면 운영체제의 page-fault handler가 필요한 처리를 한다.
- 요구 페이징에서는 필요한 페이지가 처음 접근될 때 메모리에 적재될 수 있다.

따라서 page fault는 언제나 C++ 버그를 뜻하지 않는다. 요구 페이징에서 아직 적재되지 않은 **유효한 페이지**를 처음 접근할 때도 발생할 수 있다.

반대로 C++의 dangling pointer나 out-of-bounds 접근이 항상 page fault로 발견되는 것도 아니다. 잘못된 접근 주소가 우연히 매핑된 페이지 안에 있으면 하드웨어 주소 변환은 성공할 수 있다. C++ 객체 수명과 범위 규칙은 OS의 페이지 존재 검사보다 더 세밀한 규칙이다.

5.4 서로 다른 세 가지 실패를 분리하기

현상	어느 계층의 문제인가?	의미
TLB miss	하드웨어 주소 변환 cache	변환 정보가 cache에 없어 페이지 테이블 조회 필요
유효한 페이지의 page fault	가상 메모리와 물리 상주 상태	페이지를 메모리로 가져오는 OS 처리가 필요
C++ lifetime/range 위반	C++ 객체 모델	해당 객체 또는 배열 원소를 사용할 권리가 없음

이 셋을 구분해야 “실행됐으니 유효하다”, “segmentation fault가 없으니 안전하다” 같은 잘못된 판단을 피할 수 있다.

6. 네 포인터 유형의 차이: 전이 규칙으로 보기

기본 정의는 00.메모리·객체 수명·RAII#13. 스마트 포인터에 있다. 여기서는 누가 수명을 늘리고, 어떤 연산이 소유권을 바꾸는가만 비교한다.

타입	소유권	복사	이동	대상 수명에 미치는 영향	안전한 접근 조건
<code>std::unique_ptr<T></code>	단독 소유	금지	소유권 이전	소유자가 사라지면 대상 파괴	포인터가 비어 있지 않아야 함
<code>std::shared_ptr<T></code>	공동 소유	같은 control block 공유	소유 핸들 이전	마지막 강한 소유자가 사라지면 대상 파괴	<code>shared_ptr</code> 가 비어 있지 않아야 함
<code>std::weak_ptr<T></code>	비소유 관찰	약한 관찰 복사	가능	강한 참조 수를 늘리지 않음	<code>lock()</code> 으로 얻은 <code>shared_ptr</code> 가 비어 있지 않아야 함
raw pointer <code>T*</code>	타입 자체로는 표현 안 됨	주소값 복사	주소값 이동과 의미 차이 없음	수명을 늘리거나 파괴하지 않음	별도의 수명 계약과 null·범위 조건 필요

6.1 unique_ptr : move 는 소유권 이전

PDF의 `unique_ptr` 설명은 복사를 금지하고 이동으로 유일한 소유권을 이전한다.

CPP

```
#include <memory>
#include <utility>

struct Resource {
    int id;
};

void consume(std::unique_ptr<Resource> resource) {
    // resource가 이 범위에서 소유권을 가짐
}

int main() {
    auto owner = std::make_unique<Resource>(Resource{7});
    consume(std::move(owner));

    // owner는 더 이상 객체를 소유하지 않음
}
```

함수가 `unique_ptr<T>` 를 값으로 받으면 호출자가 소유권을 넘긴다는 사실이 형식에 드러난다.

6.2 `unique_ptr::get()` : 소유권 없는 일시 접근

PDF는 `get()` 을 통해 실제 객체 주소를 얻어 일반 포인터 인자에 전달하는 예를 보여 준다.

CPP

```
#include <memory>

struct Resource {
    int id;
};

int inspect(const Resource* resource) {
    return resource == nullptr ? -1 : resource->id;
}

int main() {
    auto owner = std::make_unique<Resource>(Resource{7});
    const Resource* borrowed = owner.get();

    return inspect(borrowed) == 7 ? 0 : 1;
}
```

`borrowed` 는 객체를 삭제하지 않으며 수명을 늘리지도 않는다. `owner` 가 객체를 계속 소유하는 동안만 사용할 수 있다.

[!warning] `get()` 의 결과를 오래 보관하지 않는다

`get()` 은 소유권을 복제하지 않는다. 소유 `unique_ptr` 가 파괴되거나 이동되면 이전에 얻은 raw pointer를 계속 사용할 수 없다.

6.3 `shared_ptr` : 객체가 아니라 control block을 함께 공유

`shared_ptr` 복사본은 같은 대상과 control block을 공유한다. control block의 강한 참조 수가 마지막 소유자를 판단한다.

CPP

```
#include <memory>

struct Resource {
    int id;
};

int main() {
    auto first = std::make_shared<Resource>(Resource{7});
    auto second = first; // 같은 control block을 공유

    return first->id == second->id ? 0 : 1;
}
```

PDF는 `make_shared<T>` 가 객체와 control block을 한 번의 동적 할당으로 함께 만드는 방식을 설명한다.

6.4 같은 raw pointer로 별도 `shared_ptr` 를 만들면 안 되는 이유

다음 두 `shared_ptr` 는 같은 주소를 받았지만 서로의 존재를 모르는 별도 control block을 만든다.

CPP

```
Resource* raw = new Resource{7};

std::shared_ptr<Resource> first{raw};
// std::shared_ptr<Resource> second{raw}; // 별도 control block: 금지
```

각 control block은 자신이 마지막 소유자라고 판단할 수 있으므로 같은 객체를 두 번 삭제하는 문제가 발생한다. 최초 소유 `shared_ptr` 를 만든 뒤에는 그 `shared_ptr` 를 복사해야 한다.

CPP

```
auto first = std::make_shared<Resource>(Resource{7});
auto second = first; // 올바른 control block 공유
```

6.5 `weak_ptr` : 확인과 사용을 `lock()` 한 번으로 묶기

`weak_ptr` 는 강한 참조 수를 늘리지 않는다. 대상에 접근하려면 `lock()` 으로 임시 `shared_ptr` 를 얻는다.

CPP

```
#include <memory>

struct Resource {
    int id;
};

int read_if_alive(const std::weak_ptr<Resource>& observer) {
    if (auto owner = observer.lock()) {
        return owner->id; // 이 블록 동안 owner가 수명을 유지
    }
    return -1;
}
```

`lock()` 이 빈 `shared_ptr` 를 반환하면 대상은 이미 소멸된 상태다. 성공했다면 반환된 `shared_ptr` 가 사용 중 수명을 유지한다.

6.6 소유 그래프에서 `shared_ptr` 와 `weak_ptr`

부모가 자식을 소유하고 자식은 부모를 관찰하는 트리는 다음처럼 표현할 수 있다.

CPP

```
#include <memory>
#include <vector>

struct Node {
    std::vector<std::shared_ptr<Node>> children;
    std::weak_ptr<Node> parent;
};

int main() {
    auto parent = std::make_shared<Node>();
    auto child = std::make_shared<Node>();

    parent->children.push_back(child); // 부모가 자식을 공동 소유
    child->parent = parent;           // 자식은 부모를 관찰

    if (auto p = child->parent.lock()) {
        return p->children.size() == 1 ? 0 : 1;
    }
    return 1;
}
```

자식의 `parent` 까지 `shared_ptr` 로 만들면 부모와 자식이 서로 강하게 소유하는 순환이 될 수 있다. PDF는 이 순환에서 참조 수가 0이 되지 않아 객체가 소멸되지 않는 문제와 이를 `weak_ptr` 로 끊는 방법을 설명한다.

7. API에서 소유권을 읽는 방법

PDF의 `unique_ptr` 이동, `get()` 및 `shared_ptr` 복사 규칙을 API 선언에 적용하면 다음처럼 구분할 수 있다.

함수 선언	호출자가 읽어야 할 의미
<code>void consume(std::unique_ptr<T> value)</code>	단독 소유권을 함수에 넘김
<code>void share(std::shared_ptr<T> value)</code>	함수가 공동 소유에 참여함
<code>void observe(const T* value)</code>	선택적인 비소유 접근; 수명을 늘리지 않음
<code>void use(T& value)</code>	반드시 존재하는 객체를 빌려 사용; 소유하지 않음
<code>std::weak_ptr<T></code> 를 멤버로 저장	공동 소유 그래프를 늘리지 않는 관찰 관계

호출 목적별 선택

TEXT

함수가 객체를 파괴 시점까지 책임져야 하는가?

- └─ 단독 책임을 넘긴다 → `unique_ptr<T>`
- └─ 여러 주체가 수명을 공유 → `shared_ptr<T>`
- └─ 소유하지 않고 사용 → `T&, const T&, T*`

공동 소유 객체를 관찰하지만 수명을 늘리면 안 되는가?

- └─ `weak_ptr<T> + lock()`

raw pointer는 “나쁜 포인터”가 아니라 **소유권을 자체적으로 표현하지 않는 포인터**다. 문제는 raw pointer가 가리키는 대상의 소유자와 유효 기간을 API 계약에서 알 수 없을 때 생긴다.

8. 객체 수명과 OS 상태를 함께 추적하는 예

CPP

```
#include <cassert>
#include <memory>
#include <utility>

struct Session {
    int id;
};

int read_id(const Session* borrowed) {
    assert(borrowed != nullptr);
    return borrowed->id;
}

int finish(std::unique_ptr<Session> owned) {
    return owned->id;
}

int main() {
    auto owner = std::make_unique<Session>(Session{42});

    Session* borrowed = owner.get();
    assert(read_id(borrowed) == 42);

    assert(finish(std::move(owner)) == 42);
    assert(owner == nullptr);

    // borrowed는 더 이상 사용하지 않는다.
}
```

이 코드의 상태 변화는 다음과 같다.

시점	owner	borrowed	객체 수명
<code>make_unique</code> 직후	단독 소유	없음	살아 있음
<code>get()</code> 직후	계속 단독 소유	비소유 관찰	살아 있음
<code>std::move(owner)</code> 후	비어 있음	이전 주소값을 보유할 수 있음	<code>finish</code> 인자가 소유

시점	owner	borrowed	객체 수명
<code>finish</code> 반환 후	비어 있음	사용 불가	파괴됨

마지막 상태에서 가상 주소의 페이지가 여전히 매핑되어 있거나 같은 바이트가 남아 있을 수 있다. 그러나 C++ 객체 수명은 끝났으므로 `borrowed` 를 역참조할 수 없다. OS의 주소 변환 가능성과 C++ 객체 접근 가능성은 같은 조건이 아니다.

9. 보완된 오해 정리

“힙의 남은 용량 합계가 요청 크기보다 크면 할당된다”

항상 그렇지 않다. 외부 단편화 때문에 필요한 연속 빈 청크가 없을 수 있다.

“`delete` 하면 그 페이지가 즉시 물리 메모리에서 사라진다”

`delete` 는 C++ 객체 파괴와 저장 공간 반환의 의미를 가진다. 반환된 청크를 할당기가 다시 보유할 수 있고, OS의 페이지 상주 상태는 별도 계층에서 관리된다.

“page fault는 잘못된 포인터의 증거다”

항상 그렇지 않다. 요구 페이징에서는 유효하지만 아직 물리 메모리에 없는 페이지를 접근해도 page fault가 발생한다.

“segmentation fault가 없으면 포인터는 유효하다”

그렇지 않다. C++ 객체 수명이나 배열 범위를 위반해도 접근 주소가 매핑된 페이지 안이면 하드웨어 예외가 즉시 발생하지 않을 수 있다.

“`weak_ptr` 가 대상을 안전하게 계속 살려 둔다”

아니다. `weak_ptr` 는 강한 참조 수를 늘리지 않는다. `lock()` 이 성공해 반환한 `shared_ptr` 만 해당 작업 동안 수명을 유지한다.

“같은 raw pointer로 만든 `shared_ptr` 들은 자동으로 서로를 안다”

아니다. 별도로 생성하면 서로 다른 control block을 가질 수 있다. 기존 `shared_ptr` 를 복사해야 같은 소유권 그룹에 참여한다.

10. PDF 근거별 보완 검증표

보완 내용	cpp.pdf	ostep.pdf	ocs.pdf
RAII와 스마트 포인터의 자원 정리	676~680	-	-
<code>unique_ptr</code> 이동과 <code>get()</code>	680~688	-	-
<code>shared_ptr</code> control block과 <code>make_shared</code>	696~699	-	-
순환 참조와 <code>weak_ptr::lock()</code>	704~708	-	-
할당기 header, splitting, coalescing	-	182~186	-
내부·외부 단편화와 적합 정책	-	182~194	325쪽 이후의 연속 할당 설명
TLB hit/miss와 PTE 조회	-	248~249	325~345
요구 페이징과 page fault	-	248~253	367~375

11. 학습 점검 문제

1. `delete` 와 물리 페이지 반환이 같은 사건이 아닌 이유를 계층별로 설명할 수 있는가?
2. 빈 공간 합계는 충분하지만 할당이 실패하는 외부 단편화 예를 그릴 수 있는가?
3. splitting과 coalescing이 각각 언제 필요한지 설명할 수 있는가?
4. TLB miss와 page fault의 차이를 설명할 수 있는가?
5. page fault가 반드시 프로그램 오류가 아닌 이유를 요구 페이징과 연결할 수 있는가?
6. `unique_ptr::get()` 으로 받은 raw pointer가 소유권을 갖지 않는 이유를 설명할 수 있는가?
7. `unique_ptr` 를 값으로 받는 함수와 raw pointer를 받는 함수의 계약 차이를 설명할 수 있는가?
8. 같은 raw pointer로 두 `shared_ptr` 를 만들면 왜 double deletion으로 이어질 수 있는가?
9. `weak_ptr::lock()` 성공 후 반환된 `shared_ptr` 가 어떤 역할을 하는지 설명할 수 있는가?
10. 부모-자식 그래프에서 어느 방향을 강한 소유로 둘지 결정할 수 있는가?

12. 최종 압축 정리

1. 요청된 기본 메모리·객체 수명 항목은 00번 문서에 이미 정리되어 있다.
2. 04번은 할당기, 주소 변환, 소유권 전이라는 세 연결 지점을 보완한다.
3. 할당기는 힙의 빈 청크를 분할·병합하고 적합 정책으로 검색한다.
4. 내부 단편화는 할당 청크 안의 낭비이고, 외부 단편화는 빈 공간이 흩어진 상태다.
5. 가상 주소는 TLB와 페이지 테이블을 통해 물리 주소로 변환된다.
6. TLB miss는 변환 cache miss이고, page fault와 같은 사건이 아니다.
7. 요구 페이징의 page fault는 유효한 페이지를 처음 적재하는 정상 처리일 수 있다.
8. OS가 주소를 변환할 수 있다는 사실은 C++ 객체가 살아 있다는 보장이 아니다.
9. `unique_ptr` 는 이동으로 단독 소유권을 넘기고 `get()` 은 소유권 없는 주소만 제공한다.
10. `shared_ptr` 복사본은 같은 control block을 공유해야 한다.
11. `weak_ptr` 는 수명을 늘리지 않으며 `lock()` 에 성공한 동안만 안전하게 접근한다.
12. raw pointer는 수명을 관리하지 않으므로 소유자와 유효 기간을 별도 계약으로 보장해야 한다.

[!tip] 한 문장으로 기억하기

OS는 주소와 페이지를 관리하고, 할당기는 힙 청크를 관리하며, C++ 타입과 RAII는 객체 수명과 자원 소유권을 관리한다. 어느 한 계층의 성공도 다른 계층의 유효성을 대신 증명하지 않는다.

title: C++ 객체지향 설계와 테스트 가능한 시스템 aliases:

tags:

created: 2026-08-17 updated: 2026-08-17

- 객체지향과 인터페이스
- C++ 예외 처리와 의존성 분리
- 비동기 멀티스레드 단위 테스트
- CS
- OS
- Cpp
- OOP
- Exception
- RAII
- Async
- Multithreading
- DependencyInjection
- UnitTest

C++ 객체지향 설계와 테스트 가능한 시스템

[!summary] 한 문장 정리

업무 규칙은 작은 인터페이스 뒤에 두고, OS 자원과 스레드는 RAII 객체가 소유하며, 비동기 경계에서 수명·예외·종료를 명시하면 테스트 가능한 C++ 시스템을 만들 수 있다.

0. 이 문서의 범위

이 문서는 다음 일곱 주제를 각각 설명한 뒤 하나의 설계로 연결한다.

1. 객체지향과 인터페이스
2. 예외 처리
3. 메모리·리소스 관리
4. 비동기 처리
5. 멀티스레딩
6. 의존성 분리
7. 단위 테스트

기존 문서에서 이미 깊게 다룬 세부 원리는 필요한 만큼만 연결한다.

- 객체 수명·RAII·스마트 포인터: 00.메모리·객체 수명·RAII
- 프로세스·스레드의 자원 공유: 01.프로세스와 스레드
- 동시성과 병렬성·메모리 모델: 02.동시성과 병렬성
- mutex·condition variable·bounded queue: 03.Mutex와 Condition Variable
- 소유권·예외 안전성 보완: 04.C++ 메모리와 객체 수명 보완
- 비동기 I/O·버퍼 수명·종료 순서: 05.파일·소켓·비동기 IO와 Buffer

0.1 PDF 대조 범위

자료	직접 확인한 내용	PDF 물리 페이지
cpp.pdf	is-a/has-a, virtual dispatch, <code>override</code>	303~326쪽 부근
cpp.pdf	<code>throw</code> , <code>try / catch</code> , stack unwinding	618~629쪽 부근
cpp.pdf	<code>unique_ptr</code> , <code>shared_ptr</code> , <code>weak_ptr</code> , RAII	676~706쪽 부근
cpp.pdf	thread, mutex, condition variable, promise/future, <code>async</code> , thread pool	724~817쪽 부근
ostep.pdf	스레드와 주소 공간 공유, thread API	287~320쪽 부근
ostep.pdf	lock, condition variable, 병행성 오류	323~396쪽 부근
ostep.pdf	event loop, blocking call, AIO와 상태 관리	397~404쪽 부근
ocs.pdf	스레드와 병행성, 동기화 도구, 공유 데이터의 race condition	스레드 장과 프로세스 동기화 장

[!note] 직접 근거와 설계 종합

PDF는 C++ 언어·스레드·OS 동시성 원리를 직접 설명하지만, 특정 단위 테스트 프레임워크나 dependency injection 패턴을 독립된 장으로 다루지는 않는다. 이 문서의 의존성 분리와 단위 테스트 부분은 **PDF에서 확인한 추상화·다형성·수명·스케줄링 원칙을 실제 설계에 적용한 종합**이다.

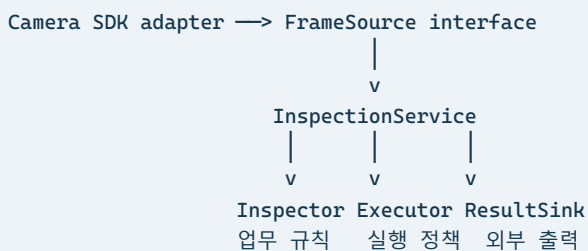
1. 전체 그림: 기능보다 경계를 먼저 본다

카메라 검사 프로그램을 예로 들면 다음 요소가 섞이기 쉽다.

- 카메라 SDK 호출
- 이미지 버퍼 소유권
- 검사 알고리즘
- 작업 큐와 스레드
- 결과 저장 또는 네트워크 전송
- 오류 기록
- 프로그램 종료

하나의 클래스가 이 모든 일을 하면 실제 장비와 스레드가 없이는 테스트하기 어렵고, 종료 순서와 예외 처리도 얽힌다. 역할을 다음처럼 나누는 편이 명확하다.

TEXT



OS 관점에서는 thread, file, socket, device handle이 구체적인 실행 자원이다. C++ 설계에서는 이 자원을 RAII adapter 안에 가두고, 업무 규칙은 작은 인터페이스만 보게 한다.

2. 객체지향과 인터페이스

2.1 객체지향의 목적

객체는 단순히 데이터와 함수를 한 클래스에 넣은 것이 아니다. 좋은 객체는 다음을 함께 가진다.

- 책임: 이 객체가 해야 하는 일
- 상태: 책임을 수행하는 데 필요한 데이터
- 불변식: public 함수 전후에 항상 만족해야 하는 조건
- 소유권: 어떤 자원의 수명을 책임지는가

예를 들어 bounded queue의 불변식은 다음처럼 표현할 수 있다.

FORMULA · CALCULATION

```
0 <= queue.size() <= capacity
closed 이후에는 새로운 작업을 받지 않음
queue가 비고 closed이면 consumer가 종료됨
```

클래스의 public API는 이 불변식을 깨뜨릴 수 있는 내부 상태를 직접 노출하지 않아야 한다.

2.2 상속은 is-a, 구성은 has-a

C++ PDF는 상속을 is-a 관계, 멤버 포함을 has-a 관계로 설명한다.

- **Manager is an Employee** : 대체 가능성이 성립하면 public 상속 후보
- **Car has an Engine** : 기능을 소유하거나 사용한다면 구성 후보

코드 재사용만을 위해 상속하면 기반 클래스의 내부 변화와 파생 클래스가 강하게 결합된다. 실행기, 카메라, 저장소 같은 협력 객체는 대개 has-a 구성과 의존성 주입이 적합하다.

2.3 인터페이스는 교체 가능한 계약이다

```

CPP
#include <stdint>
#include <vector>

struct Frame {
    std::uint64_t id{};
    std::vector<unsigned char> pixels;
};

struct InspectionResult {
    std::uint64_t frame_id{};
    bool passed{};
};

class IInspector {
public:
    virtual ~IInspector() = default;
    virtual InspectionResult inspect(const Frame& frame) = 0;
};

```

이 인터페이스가 약속하는 것은 “프레임을 받아 결과를 만든다”는 행위다. 카메라 SDK, 파일 시스템, mutex 같은 구현 세부는 포함하지 않는다.

2.4 다형성에서 지켜야 할 C++ 규칙

- 기반 클래스 포인터로 파생 객체를 삭제할 수 있다면 destructor를 **virtual** 로 만든다.
- 재정의 함수에는 **override** 를 붙여 signature 실수를 컴파일 시간에 잡는다.
- 인터페이스에는 필요한 최소 행위만 둔다.
- 객체를 기반 클래스 값으로 복사하면 slicing이 발생하므로 보통 reference나 pointer로 다룬다.
- 소유하지 않는 의존성은 reference 또는 명시적인 observer pointer로 표현한다.
- 소유하는 다형 객체는 보통 **std::unique_ptr<Base>** 로 표현한다.

```

CPP
class IFrameSource {
public:
    virtual ~IFrameSource() = default;
    virtual Frame next() = 0;
};

class CameraFrameSource final : public IFrameSource {
public:
    Frame next() override;
};

```

final 은 이 구현을 다시 상속해 동작을 바꾸는 것이 설계 의도가 아님을 나타낸다.

2.5 좋은 인터페이스의 조건

조건	질문
작음	사용하는 쪽에 필요하지 않은 함수까지 강요하는가?
의미 중심	OS 함수 이름을 그대로 복사했는가, 업무 의미를 표현하는가?
소유권 명확	인수와 반환값 중 누가 무엇을 소유하는가?
오류 계약 명확	실패를 예외, 값, 상태 코드 중 무엇으로 표현하는가?
thread 계약 명확	동시에 호출해도 되는가? callback thread는 누구 것인가?
종료 계약 명확	<code>stop()</code> 은 요청만 하는가, 완료까지 기다리는가?

3. 예외 처리

3.1 예외는 정상 결과와 다른 실패 경로다

C++ PDF는 `throw` 뒤 가장 가까운 호출 `catch` 까지 제어가 이동하고, 그 사이 stack 객체의 destructor가 호출되는 stack unwinding을 설명한다.

예외는 다음 상황에 잘 맞는다.

- 현재 함수가 정상 결과를 만들 수 없음
- 호출자가 더 높은 수준에서 복구 정책을 결정해야 함
- 생성자가 유효한 객체를 만들 수 없음

다음은 예외보다 명시적 결과가 더 적합할 수 있다.

- 파일 끝, 검색 결과 없음처럼 예상되는 분기
- 매우 빈번하게 발생하는 정상적인 재시도 상태
- ABI나 언어 경계를 넘는 C callback
- 실시간 경로에서 예외 사용이 프로젝트 정책상 금지된 경우

3.2 예외와 오류 코드를 섞지 않는다

한 추상화 계층 안에서는 실패 표현을 일관되게 정한다.

CPP

```
Frame read_frame(); // 실패 시 예외
std::optional<Frame> try_read_frame(); // 없음이 정상 상태
std::expected<Frame, ReadError> read(); // C++23: 오류를 값으로 표현
```

함수 이름과 반환형만 보고도 계약을 이해할 수 있어야 한다.

3.3 RAII가 예외 안전성의 기반이다

CPP

```
void process() {
    std::vector<std::byte> buffer(4096);
    std::lock_guard lock(mutex);
    update(buffer); // 예외가 나도 lock과 buffer가 정리됨
}
```

수동 `lock()` / `unlock()`, `new` / `delete`, `open` / `close` 쌍은 중간 `throw` 나 조기 반환이 생길 때 쉽게 깨진다. 자원 획득 직후 소유 RAII 객체에 넣으면 모든 탈출 경로가 같은 정리를 사용한다.

3.4 예외 안전성 보장 수준

수준	의미	구현 전략
no-throw	연산이 예외를 외부로 내보내지 않음	destructor, <code>swap</code> , cleanup
strong guarantee	실패하면 관찰 가능한 상태가 호출 전과 같음	임시 객체에 완성한 뒤 commit/swap
basic guarantee	실패해도 불변식과 자원 안전은 유지	RAII, 부분 상태의 유효성 유지
no guarantee	실패 뒤 상태를 신뢰할 수 없음	피해야 함

CPP

```
void Config::replace(std::vector<Item> next) {
    validate(next); // 실패하면 기존 items_는 그대로
    items_.swap(next); // 검증 완료 뒤 commit
}
```

3.5 destructor와 예외

destructor는 원칙적으로 예외를 밖으로 내보내지 않아야 한다. stack unwinding 도중 destructor가 다시 예외를 던지면 프로그램이 종료될 수 있다.

CPP

```
class FileLog {
public:
    ~FileLog() noexcept {
        try {
            flush_if_needed();
        } catch (...) {
            // destructor에서는 기록 가능한 최소 수단을 사용하고 삼킨다.
        }
    }
};
```

중요한 저장 실패를 destructor에서 처음 발견하도록 설계하지 말고, 호출자가 결과를 확인할 수 있는 명시적 `flush()` / `close()` 를 제공한다.

3.6 생성자에서 예외가 발생할 때

생성자가 끝나기 전에 예외가 발생하면 아직 완성되지 않은 객체 자신의 destructor는 호출되지 않는다. 다만 이미 완전히 생성된 base와 member 객체는 역순으로 파괴된다. 따라서 생성자 본문에서 raw 자원을 먼저 획득하고 나중에 RAII 객체에 넣지 말고, 자원 자체를 RAII member로 즉시 구성한다.

CPP

```
class Session {
public:
    explicit Session(const Path& path)
        : file_(open_file(path)), buffer_(4096) {
        validate_header(file_); // 실패해도 file_과 buffer_는 정리됨
    }

private:
    UniqueFile file_;
    std::vector<std::byte> buffer_;
};
```

3.7 비동기 경계의 예외

스레드 함수 밖으로 예외가 빠져나가면 `std::terminate` 로 이어질 수 있다. 비동기 작업은 다음 중 하나로 예외를 전달해야 한다.

- `promise.set_exception()` 과 `future.get()`
- 작업 결과의 error field
- worker 최상단의 `catch` 에서 오류 sink로 변환

CPP

```
auto future = std::async(std::launch::async, [] {
    return risky_work();
});

try {
    auto value = future.get(); // 저장된 예외가 여기서 다시 던져짐
} catch (const std::exception& error) {
    report(error.what());
}
```

4. 메모리·리소스 관리

4.1 메모리보다 리소스가 더 넓다

RAII가 관리하는 대상은 heap 메모리에 한정되지 않는다.

- 파일과 socket handle
- mutex lock
- thread
- 카메라·stream handle

- SDK frame lease
- memory mapping
- transaction

OS는 프로세스가 끝나면 여러 커널 자원을 회수하지만, 정상 실행 중의 파일 flush, frame 반납, lock 해제, thread 종료 순서까지 대신 설계해 주지는 않는다.

4.2 소유권 타입 선택

관계	기본 표현	의미
값 자체 소유	값 멤버, <code>std::vector</code> , <code>std::string</code>	객체와 함께 생성·소멸
단독 동적 소유	<code>std::unique_ptr<T></code>	복사 금지, 이동으로 소유권 이전
공동 수명	<code>std::shared_ptr<T></code>	마지막 소유자가 파괴
공동 소유 관찰	<code>std::weak_ptr<T></code>	참조 수를 늘리지 않음
비소유 참조	<code>T&</code> , <code>T*</code> , <code>std::span<T></code>	대상의 수명을 연장하지 않음
OS/SDK handle	전용 RAII wrapper	정확한 close/release 함수 호출

기본 선택은 값 또는 `unique_ptr` 다. `shared_ptr` 는 정말 공동 수명이 필요할 때만 사용한다. `shared_ptr` 는 thread safety 전체를 보장하지 않고 제어 블록의 수명 관리만 제공한다.

4.3 Rule of Zero

멤버가 스스로 자원을 관리하면 사용자 정의 destructor·복사·이동 연산을 작성하지 않아도 된다.

```

CPP
class FrameBatch {
public:
    std::vector<Frame> frames;
    std::string source_name;
}; // compiler가 안전한 복사·이동·소멸을 구성
  
```

raw handle을 직접 소유해야 한다면 작은 RAII wrapper 하나에만 그 책임을 모으고, 상위 클래스는 그 wrapper를 값으로 보유한다.

4.4 인터페이스와 소유권을 함께 설계한다

```

CPP
class InspectionService {
public:
    InspectionService(IInspector& inspector, IResultSink& sink);

private:
    IInspector& inspector_; // 소유하지 않음: service보다 오래 살아야 함
    IResultSink& sink_;     // 소유하지 않음
};
  
```

reference 주입은 null이 아니며 소유하지 않는다는 뜻이 분명하지만, 외부 객체가 service보다 오래 살아야 한다. service가 의존성을 소유해야 한다면 `std::unique_ptr<IInspector>` 로 받아 이동한다.

CPP

```
class InspectionService {
public:
    explicit InspectionService(std::unique_ptr<IInspector> inspector)
        : inspector_(std::move(inspector)) {}

private:
    std::unique_ptr<IInspector> inspector_;
};
```

5. 비동기 처리

5.1 비동기는 실행 정책이다

비동기는 호출한 작업의 완료를 지금 기다리지 않고 나중에 관찰하는 구조다. 반드시 병렬 실행을 의미하지는 않는다.

개념	핵심 질문
동기	호출이 반환될 때 작업이 끝났는가?
비동기	완료를 나중에 callback/event/future로 받는가?
동시성	여러 작업의 진행 기간이 겹치는가?
병렬성	여러 작업이 같은 순간에 실제 실행되는가?

`std::async` 도 launch policy에 따라 새 실행 주체에서 실행되거나 `future.get()` 시점까지 deferred될 수 있다. 실행 시점을 중요하게 다룬다면 policy를 명시한다.

5.2 비동기 API가 추가하는 상태

동기 함수는 호출 stack이 진행 상태를 보관한다. 이벤트·callback 방식에서는 프로그램이 상태를 별도로 보관해야 한다.

- 요청 식별자
- input buffer와 output buffer
- 현재 단계
- 완료·오류·취소 상태
- callback 대상의 수명
- timeout 또는 deadline

OSTEP의 AIO 예처럼 요청 구조체가 buffer 주소와 길이를 가리킨다면 완료 또는 취소 완료까지 그 메모리가 유효해야 한다.

5.3 callback capture 수명

C++

```
class Session {
public:
    void start() {
        executor_.post([this] { use_members(); });
    }
};
```

작업 실행 전에 `Session` 이 파괴되면 `this` 는 dangling pointer가 된다. 해결 방법은 계약에 따라 다르다.

- 소유 객체의 destructor가 executor를 stop/join하여 작업보다 오래 살아 있게 함
- 값으로 필요한 데이터만 capture
- 공동 비동기 수명이 필요하면 `shared_ptr` / `weak_ptr` 를 신중히 사용
- callback 등록 해제와 in-flight callback 종료 대기 제공

C++

```
std::weak_ptr<Session> weak = shared_from_this();
executor.post([weak] {
    if (auto self = weak.lock()) {
        self->use_members();
    }
});
```

이 코드는 use-after-free는 피하지만 작업이 조용히 취소되는 의미가 적합한지도 별도로 판단해야 한다.

5.4 취소는 하나의 프로토콜이다

`cancel()` 호출과 작업이 실제로 멈춘 시점은 다를 수 있다.

TEXT

CancelRequested

- > 작업이 요청을 관찰
- > OS/SDK 작업 취소 또는 정상 종료
- > Completion/Cancelled 통지
- > buffer와 요청 객체 해제 가능

취소는 성공·실패와 함께 비동기 연산의 정상적인 종결 상태로 모델링한다.

6. 멀티스레딩

6.1 OS 관점의 핵심

같은 프로세스의 스레드는 주소 공간과 heap을 공유하지만 register, program counter, stack은 따로 가진다. 공유 메모리는 통신을 쉽게 하지만 동시에 race condition의 원인이 된다.

TEXT

Process address space

code	shared
globals/static	shared
heap	shared
Thread A stack	private
Thread B stack	private

동일한 메모리에 둘 이상의 스레드가 접근하고, 그중 하나 이상이 쓰며, 동기화가 없다면 data race가 될 수 있다. C++의 data race는 undefined behavior다.

6.2 mutex는 데이터가 아니라 불변식을 보호한다

CPP

```
class Counter {
public:
    void increment() {
        std::lock_guard lock(mutex_);
        ++value_;
    }

    int value() const {
        std::lock_guard lock(mutex_);
        return value_;
    }

private:
    mutable std::mutex mutex_;
    int value_ = 0;
};
```

“value_에 mutex를 붙였다”보다 “Counter의 모든 public 연산이 value_의 일관성을 유지한다”고 이해하는 것이 좋다. 여러 필드가 하나의 불변식을 구성한다면 같은 lock 아래에서 함께 갱신한다.

6.3 condition variable은 predicate와 한 쌍이다

CPP

```
std::unique_lock lock(mutex_);
condition_.wait(lock, [this] {
    return closed_ || !queue_.empty();
});
```

- 상태는 mutex 아래에서 검사·변경한다.
- wakeup 자체가 조건의 참을 보장하지 않으므로 predicate를 다시 검사한다.
- 종료 상태도 predicate에 포함한다.
- queue에서 꺼낸 작업은 가능하면 lock을 해제한 뒤 실행한다.

6.4 thread도 소유 자원이다

thread를 시작한 객체는 다음을 책임져야 한다.

- 새 작업 접수를 언제 막는가?
- 대기 중인 worker를 어떻게 깨우는가?
- queue를 drain할지 discard할지?
- 누가 `join()` 하는가?
- worker가 사용하는 의존성은 언제 파괴되는가?

FORMULA · CALCULATION

```

accepting = false
-> wait 중인 worker notify
-> queue drain/discard
-> worker loop 종료
-> join
-> worker가 참조하던 의존성 파괴

```

6.5 lock의 범위를 외부 코드까지 넓히지 않는다

interface의 가상 함수나 사용자 callback을 mutex를 잡은 채 호출하면 다음 위험이 있다.

- 상대 코드가 다시 나를 호출해 deadlock
- 느린 I/O로 critical section 장기 점유
- 예상하지 못한 예외가 lock 범위와 상태 전이를 복잡하게 함

공유 상태에서 필요한 값을 복사·이동해 꺼내고 lock을 해제한 뒤 외부 코드를 호출한다.

7. 의존성 분리

7.1 의존성 역전의 실제 의미

상위 업무 정책이 하위 SDK의 구체 타입과 함수 이름을 직접 알지 않게 만든다.

TEXT

나쁜 방향

InspectionService -> VendorCamera -> VendorBuffer -> VendorThread

분리한 방향

InspectionService -> IFrameSource / IExecutor / IResultSink

```

      ^           ^           ^
      |           |           |
  CameraAdapter ThreadExecutor FileResultSink

```

상위 정책과 adapter 모두 상위에서 정의한 작은 계약에 의존한다.

7.2 무엇을 분리해야 하는가?

- 외부 장치·SDK
- 파일·socket·database
- system clock와 timeout
- random 값
- thread/executor
- 환경 변수와 설정 파일
- logger와 결과 출력

순수 계산까지 모두 virtual interface로 만들 필요는 없다. 바뀌는 속도와 테스트 경계를 기준으로 분리한다.

7.3 런타임 다형성과 compile-time 주입

방식	예	장점	비용/제약
virtual interface	<code>IInspector&</code>	ABI 경계, 런타임 교체, 이해하기 쉬운 fake	간접 호출, 수명 계약 필요
template	<code>template<class Executor></code>	inlining, 정적 검증, zero-overhead 가능	구현 노출, compile time 증가
callable	<code>std::function</code> , lambda	작은 전략 주입에 간단	type erasure 비용, 소유 capture 주의

장치 adapter처럼 구현을 런타임에 바꿀 필요가 있으면 virtual interface가 자연스럽다. 작은 clock 함수나 정책은 template/lambda가 더 간단할 수 있다.

7.4 생성자가 유효한 상태를 완성한다

필수 의존성은 setter로 나중에 넣기보다 생성자에서 받는다.

CPP

```

InspectionService(
    IInspector& inspector,
    IExecutor& executor,
    IResultSink& sink)
: inspector_(inspector), executor_(executor), sink_(sink) {}

```

이렇게 하면 null 여부, 초기화 순서, 테스트 대체 구현이 명확해진다.

8. 단위 테스트

8.1 단위 테스트의 목적

단위 테스트는 함수 호출 횟수를 많이 확인하는 것이 목적이 아니다. 외부 환경과 독립적으로 중요한 규칙과 상태 전이를 빠르고 반복 가능하게 확인하는 것이 목적이다.

좋은 테스트는 다음 성질을 가진다.

- 결정적: scheduler 운에 따라 성공·실패하지 않음
- 격리됨: 실제 카메라, 네트워크, wall clock이 없어도 실행
- 빠름: sleep과 큰 timeout에 의존하지 않음
- 행위 중심: 구현 세부보다 결과와 불변식을 검증
- 실패 원인이 분명함

8.2 fake, stub, mock의 역할

대역	목적	예
stub	준비된 값을 반환	항상 같은 frame을 주는 source
fake	단순하지만 실제로 동작하는 구현	memory sink, inline executor
spy	호출 내용을 기록	publish된 result 목록
mock	기대한 상호작용을 검증	특정 순서의 release 호출

프레임 결과 자체가 중요하다면 spy에 저장된 결과를 검증한다. 구현 내부 함수의 정확한 호출 순서를 과도하게 고정하면 refactoring이 어려워진다.

8.3 thread를 제거해 업무 규칙을 테스트한다

실서비스에서는 thread pool을 쓰더라도 테스트에서는 즉시 실행하는 executor를 주입할 수 있다.

CPP

```
class InlineExecutor final : public IExecutor {
public:
    void post(std::function<void()> task) override {
        task();
    }
    void stop() noexcept override {}
};
```

이렇게 하면 검사 제출과 결과 수집이 한 thread에서 끝나므로 sleep이나 polling 없이 assert할 수 있다. 이것이 의존성 분리가 테스트 가능성을 만드는 대표적인 예다.

8.4 멀티스레드 구현 자체는 별도로 테스트한다

업무 규칙 테스트에서 thread를 제거했다고 thread 안전성 검증이 끝나는 것은 아니다. executor, queue, shutdown 구현에는 별도의 component test가 필요하다.

- 동시에 여러 producer가 제출해도 작업이 유실·중복되지 않는가?
- `stop()` 과 `post()` 가 경쟁할 때 계약대로 동작하는가?
- queue가 빈 상태에서 stop하면 worker가 깨어나는가?
- 작업이 예외를 던져도 worker가 정한 정책대로 계속 동작하는가?
- 모든 작업이 끝난 뒤 thread가 join되는가?
- ThreadSanitizer 같은 도구에서 data race가 보고되지 않는가?

8.5 시간 기반 테스트에서 sleep을 피한다

CPP

```
class IClock {  
public:  
    virtual ~IClock() = default;  
    virtual TimePoint now() const = 0;  
};
```

fake clock을 주입해 시간을 직접 전진시키면 “100 ms를 실제로 기다린 뒤 확인”하는 느리고 불안정한 테스트를 피할 수 있다.

9. 통합 C++20 예제

다음 예제는 업무 규칙, 실행 정책, 결과 출력을 분리한다. 실제 환경은 `ThreadExecutor`, 단위 테스트는 `InlineExecutor` 를 사용한다.

CPP · 1/6

```
#include <cassert>
#include <condition_variable>
#include <cstdint>
#include <deque>
#include <functional>
#include <mutex>
#include <stdexcept>
#include <string>
#include <thread>
#include <utility>
#include <vector>

struct Frame {
    std::uint64_t id{};
    std::vector<unsigned char> pixels;
};

struct InspectionResult {
    std::uint64_t frame_id{};
    bool passed{};
    std::string error;
};

class IInspector {
public:
    virtual ~IInspector() = default;
    virtual InspectionResult inspect(const Frame& frame) = 0;
};

class IResultSink {
public:
    virtual ~IResultSink() = default;
    virtual void publish(InspectionResult result) = 0;
};

class IExecutor {
public:
    virtual ~IExecutor() = default;
    virtual void post(std::function<void()> task) = 0;
    virtual void stop() noexcept = 0;
};
```

CPP · 2/6

```

class InspectionService {
public:
    InspectionService(
        IInspector& inspector,
        IExecutor& executor,
        IResultSink& sink)
        : inspector_(inspector), executor_(executor), sink_(sink) {}

    ~InspectionService() {
        stop();
    }

    InspectionService(const InspectionService&) = delete;
    InspectionService& operator=(const InspectionService&) = delete;

    void submit(Frame frame) {
        {
            std::lock_guard lock(state_mutex_);
            if (!accepting_) {
                throw std::runtime_error("service is stopped");
            }
        }

        executor_.post([this, frame = std::move(frame)]() mutable {
            InspectionResult result;
            try {
                result = inspector_.inspect(frame);
            } catch (const std::exception& error) {
                result = InspectionResult{frame.id, false, error.what()};
            } catch (...) {
                result = InspectionResult{frame.id, false, "unknown error"};
            }
            sink_.publish(std::move(result));
        });
    }

    void stop() noexcept {
        {
            std::lock_guard lock(state_mutex_);
            if (!accepting_) {
                return;
            }
            accepting_ = false;
        }
    }
}

```

CPP · 3/6

```

        executor_.stop(); // 이 계약에서는 queue drain과 join까지 완료한다.
    }

private:
    IInspector& inspector_;
    IExecutor& executor_;
    IResultSink& sink_;
    std::mutex state_mutex_;
    bool accepting_ = true;
};

class ThreadExecutor final : public IExecutor {
public:
    ThreadExecutor() : worker_([this] { run(); }) {}

    ~ThreadExecutor() override {
        stop();
    }

    void post(std::function<void()> task) override {
        {
            std::lock_guard lock(mutex_);
            if (!accepting_) {
                throw std::runtime_error("executor is stopped");
            }
            tasks_.push_back(std::move(task));
        }
        ready_.notify_one();
    }

    void stop() noexcept override {
        {
            std::lock_guard lock(mutex_);
            accepting_ = false;
        }
        ready_.notify_all();
        if (worker_.joinable()) {
            worker_.join();
        }
    }
}

```

CPP · 4/6

```

private:
    void run() noexcept {
        for (;;) {
            std::function<void()> task;
            {
                std::unique_lock lock(mutex_);
                ready_.wait(lock, [this] {
                    return !accepting_ || !tasks_.empty();
                });

                if (tasks_.empty()) {
                    return;
                }

                task = std::move(tasks_.front());
                tasks_.pop_front();
            }

            try {
                task();
            } catch (...) {
                // worker 경계 정책: task 예외가 thread 밖으로 나가지 않게 한다.
            }
        }
    }

    std::mutex mutex_;
    std::condition_variable ready_;
    std::deque<std::function<void()>> tasks_;
    bool accepting_ = true;
    std::thread worker_;
};

class InlineExecutor final : public IExecutor {
public:
    void post(std::function<void()> task) override {
        if (stopped_) {
            throw std::runtime_error("executor is stopped");
        }
        task();
    }

    void stop() noexcept override {
        stopped_ = true;
    }
}

```

CPP · 5/6

```

private:
    bool stopped_ = false;
};

class ThresholdInspector final : public IInspector {
public:
    explicit ThresholdInspector(unsigned char minimum) : minimum_(minimum) {}

    InspectionResult inspect(const Frame& frame) override {
        if (frame.pixels.empty()) {
            throw std::invalid_argument("empty frame");
        }

        const bool passed = frame.pixels.front() >= minimum_;
        return InspectionResult{frame.id, passed, {}};
    }

private:
    unsigned char minimum_{};
};

class CollectingSink final : public IResultSink {
public:
    void publish(InspectionResult result) override {
        std::lock_guard lock(mutex_);
        results_.push_back(std::move(result));
    }

    std::vector<InspectionResult> snapshot() const {
        std::lock_guard lock(mutex_);
        return results_;
    }

private:
    mutable std::mutex mutex_;
    std::vector<InspectionResult> results_;
};

void unit_test_success_without_thread() {
    ThresholdInspector inspector{100};
    InlineExecutor executor;
    CollectingSink sink;
    InspectionService service{inspector, executor, sink};
}

```

CPP · 6/6

```

    service.submit(Frame{7, {120}});

    const auto results = sink.snapshot();
    assert(results.size() == 1);
    assert(results[0].frame_id == 7);
    assert(results[0].passed);
    assert(results[0].error.empty());
}

void unit_test_exception_becomes_result() {
    ThresholdInspector inspector{100};
    InlineExecutor executor;
    CollectingSink sink;
    InspectionService service{inspector, executor, sink};

    service.submit(Frame{8, {}});

    const auto results = sink.snapshot();
    assert(results.size() == 1);
    assert(results[0].frame_id == 8);
    assert(!results[0].passed);
    assert(results[0].error == "empty frame");
}

void component_test_real_worker() {
    ThresholdInspector inspector{100};
    ThreadExecutor executor;
    CollectingSink sink;
    InspectionService service{inspector, executor, sink};

    service.submit(Frame{9, {130}});
    service.stop(); // queue drain과 join 뒤 반환

    const auto results = sink.snapshot();
    assert(results.size() == 1);
    assert(results[0].passed);
}

int main() {
    unit_test_success_without_thread();
    unit_test_exception_becomes_result();
    component_test_real_worker();
}

```

9.1 예제의 소유권과 수명

- `Frame` 은 pixel buffer를 값으로 소유하며 task lambda로 이동된다.
- `service`는 세 의존성을 소유하지 않으므로 의존성이 `service`보다 오래 살아야 한다.
- 예제의 지역 변수 선언 순서상 `service`가 먼저 파괴되고 `sink`, `executor`, `inspector`가 나중에 파괴된다.
- `service`의 `stop()` 은 `executor`의 queue drain과 worker join까지 완료한다.
- worker가 join된 뒤에만 `service`와 의존성을 파괴하므로 lambda의 `this` 가 dangling되지 않는다.
- `CollectingSink` 는 실제 worker test에서도 안전하도록 자체 mutex를 가진다.

9.2 예제에서 예외를 두 번 막는 이유

`InspectionService` 는 검사 알고리즘 예외를 업무 결과로 바꾼다. `ThreadExecutor` 의 최상위 `catch` 는 그 밖의 예외가 thread 함수 밖으로 탈출하지 못하게 하는 마지막 방어선이다.

실제 시스템에서는 마지막 방어선에서 오류를 무시하기보다 다음을 기록해야 한다.

- task 식별자
- exception message
- 실패 시간과 thread 정보
- queue를 계속 처리할지 중지할지 정책

10. 설계 선택과 trade-off

10.1 interface가 많을수록 좋은가?

아니다. 다음 경우에 경계를 만든다.

- 외부 시스템에 접근한다.
- 테스트에서 대체해야 한다.
- 구현이 둘 이상 존재한다.
- 실행 정책과 업무 정책을 독립적으로 바꾸고 싶다.
- 수명·threading·오류 계약을 한곳에 캡슐화해야 한다.

단순한 값 객체나 내부 계산 함수까지 모두 interface로 만들면 탐색 비용과 간접 계층만 늘어난다.

10.2 예외와 `expected` 중 무엇을 쓰는가?

- 실패가 드물고 여러 호출 계층을 건너 정책 지점까지 전달되어야 하면 예외가 편리하다.
- 실패가 정상적인 분기이고 호출자가 즉시 처리해야 하면 `expected` /오류 값이 명시적이다.
- C callback·SDK 경계를 넘기 전에는 예외를 catch해 상태 코드로 변환한다.
- 어느 방식을 택하든 RAII로 자원 안전을 유지한다.

10.3 virtual interface의 성능이 걱정될 때

먼저 실제 측정으로 병목인지 확인한다. 이미지 처리처럼 큰 연산을 한 번 호출하는 경계에서는 가상 호출 한 번보다 buffer 복사·cache miss·알고리즘 비용이 훨씬 클 수 있다. 작은 함수가 매우 빈번한 hot loop라면 template 정책이나 값 타입을 검토한다.

11. 흔한 실패 패턴

11.1 God object

한 클래스가 SDK 연결, frame 수신, 검사, 파일 저장, thread 종료를 모두 담당한다. 작은 변경이 전체 상태 머신에 영향을 주고 실제 장비 없이는 테스트하기 어렵다.

11.2 interface가 raw pointer 소유권을 숨김

CPP

```
virtual Frame* next() = 0; // 누가 delete하는가? SDK buffer인가?
```

값 반환, `unique_ptr`, non-owning view와 명시적 lease 중 하나로 계약을 분명히 한다.

11.3 constructor에서 thread를 시작한 뒤 파생 virtual 함수 호출

완전히 생성되지 않은 객체를 다른 thread가 관찰할 수 있다. construction과 `start()` 를 분리하거나, 모든 불변식이 완성된 factory에서 시작한다.

11.4 destructor에서 join 없는 `std::thread`

joinable thread의 destructor는 프로그램 종료를 일으킬 수 있다. thread 소유 객체는 stop/join 정책을 가져야 한다.

11.5 sleep으로 동시성 테스트

CPP

```
service.submit(frame);
std::this_thread::sleep_for(100ms);
assert(done);
```

느린 CI나 scheduler 상황에 따라 실패한다. inline executor, latch, future, condition variable처럼 완료를 의미하는 동기화 수단을 사용한다.

11.6 mutex를 잡고 callback 호출

callback 구현을 알 수 없으므로 재진입 deadlock과 장시간 lock 점유가 생긴다. 내부 상태를 확인한 뒤 lock을 풀고 callback을 호출한다.

11.7 `shared_ptr` 로 모든 수명 문제를 덮음

소유자가 불명확해지고 순환 참조와 예측하기 어려운 파괴 thread가 생긴다. 기본은 단일 소유이며 observer에는 reference나 `weak_ptr` 를 사용한다.

12. 실무 체크리스트

객체지향·인터페이스

- ☐ 클래스마다 한 문장으로 설명 가능한 책임이 있는가?
- ☐ public API가 불변식을 보호하는가?
- ☐ 상속 관계가 진짜 is-a와 대체 가능성을 만족하는가?
- ☐ 기반 interface에 virtual destructor가 있는가?
- ☐ override 함수에 `override` 를 사용했는가?

예외·리소스

- ☐ 실패 표현이 예외와 오류 값 중 하나로 일관적인가?
- ☐ 획득 직후 RAII 객체가 자원을 소유하는가?
- ☐ destructor가 예외를 밖으로 내보내지 않는가?
- ☐ 부분 갱신 실패 뒤에도 불변식이 유지되는가?
- ☐ OS/SDK 자원은 정확한 close/release 함수로 정리되는가?

비동기·멀티스레드

- ☐ task가 참조하는 객체와 buffer가 완료까지 살아 있는가?
- ☐ 예외가 worker thread 밖으로 빠져나가지 않는가?
- ☐ shared state와 이를 보호하는 lock이 문서화되어 있는가?
- ☐ condition variable의 predicate에 종료 상태가 포함되는가?
- ☐ stop, drain/discard, join, dependency destroy 순서가 정해져 있는가?

의존성·테스트

- ☐ 업무 규칙이 카메라 SDK·파일·socket을 직접 호출하지 않는가?
- ☐ executor, clock, 외부 출력이 테스트 대역으로 교체 가능한가?
- ☐ 단위 테스트가 sleep과 실제 장비 없이 결정적으로 실행되는가?
- ☐ thread 구현 자체의 component/stress test가 별도로 있는가?
- ☐ 결과와 불변식을 검증하고 내부 구현 순서를 과도하게 고정하지 않는가?

13. 핵심 결론

1. 객체지향의 핵심은 클래스 수가 아니라 책임·불변식·소유권의 경계다.
2. interface는 구현 교체보다 먼저 계약과 의존 방향을 명확하게 만든다.
3. 예외 안전성은 `catch` 의 개수가 아니라 stack unwinding 중에도 자원이 정리되는 RAII에서 시작한다.
4. 메모리뿐 아니라 thread, file, socket, SDK frame도 소유 자원이다.
5. 비동기 코드는 요청 상태, callback 수명, 취소 완료, 오류 전달을 명시해야 한다.
6. 멀티스레드의 핵심은 thread 생성이 아니라 공유 상태와 종료 protocol이다.
7. executor와 외부 adapter를 분리하면 업무 규칙을 thread·장비 없이 테스트할 수 있다.
8. 단위 테스트와 실제 thread component test는 목적이 다르므로 분리한다.

14. 작성 후 검증 결과

14.1 Markdown·코드 검증

- UTF-8 replacement character가 없는지 확인했다.
- frontmatter, 제목 계층과 모든 code fence의 짝을 확인했다.
- 00.메모리·객체 수명·RAII부터 05.파일·소켓·비동기 IO와 Buffer까지 내부 링크 대상이 모두 존재함을 확인했다.
- 요청한 일급 주제가 각각 독립된 절과 통합 예제에 포함되어 있는지 확인했다.
- 통합 예제를 MSVC C++20 `/W4 /permissive-` 로 컴파일했다.
- 두 단위 테스트와 실제 worker component test를 포함한 실행 파일을 50회 반복 실행했다.

14.2 PDF 2차 재대조

문서의 주장	다시 확인한 원문	결과
상속은 is-a 관계이며 virtual 함수는 실제 객체에 따라 동적 binding된다	cpp.pdf 물리 303~320쪽	일치
<code>override</code> 가 잘못된 재정의를 compile time에 찾는다	cpp.pdf 물리 320~322쪽	일치
예외 전파 중 stack 객체의 destructor가 호출된다	cpp.pdf 물리 621~626쪽	일치
생성자 실패 시 완성되지 않은 객체 자신의 destructor는 호출되지 않는다	cpp.pdf 물리 626쪽	일치, RAII member 설명 보완
<code>unique_ptr</code> 는 단일 소유와 RAII를, <code>shared_ptr</code> 는 공동 수명을 표현한다	cpp.pdf 물리 676~705쪽	일치
<code>future</code> 는 비동기 결과·예외의 관찰 지점이 되고 <code>async</code> 정책에 따라 실행 방식이 달라진다	cpp.pdf 물리 796~810쪽	일치
thread pool은 공유 queue를 mutex/CV로 보호하고 stop-notify:join한다	cpp.pdf 물리 812~817쪽	일치
같은 프로세스의 thread는 주소 공간을 공유하고 stack은 따로 가진다	ostep.pdf 물리 287쪽, ocs.pdf 스레드 장	일치
condition variable은 조건을 기다리고 상태 변화에 따라 깨운다	ostep.pdf 물리 348쪽	일치
event/AIO는 호출 stack 밖에 요청·buffer·완료 상태를 보관해야 한다	ostep.pdf 물리 400~401쪽	일치
의존성 주입과 unit-test double의 구체 패턴	세 PDF에 독립된 직접 설명 없음	PDF 원리에서 도출한 설계 종합으로 명시

2차 대조 후 특정 테스트 프레임워크의 동작을 PDF 근거처럼 서술한 부분이 없음을 확인했다. 또한 생성자 예외와 RAII member의 관계를 보완하고, 비동기·멀티스레드 종료 계약을 통합 예제의 소유권 분석과 연결했다.

참고한 PDF

- [cpp.pdf](#) : 클래스 상속과 다형성, 예외 처리, 스마트 포인터, C++ thread와 비동기 도구
- [ostep.pdf](#) : 스레드 API, lock과 condition variable, 병행성 오류, 이벤트 기반 병행성과 AIO
- [ocs.pdf](#) : 스레드와 병행성, 프로세스 동기화, mutex와 공유 데이터 제어